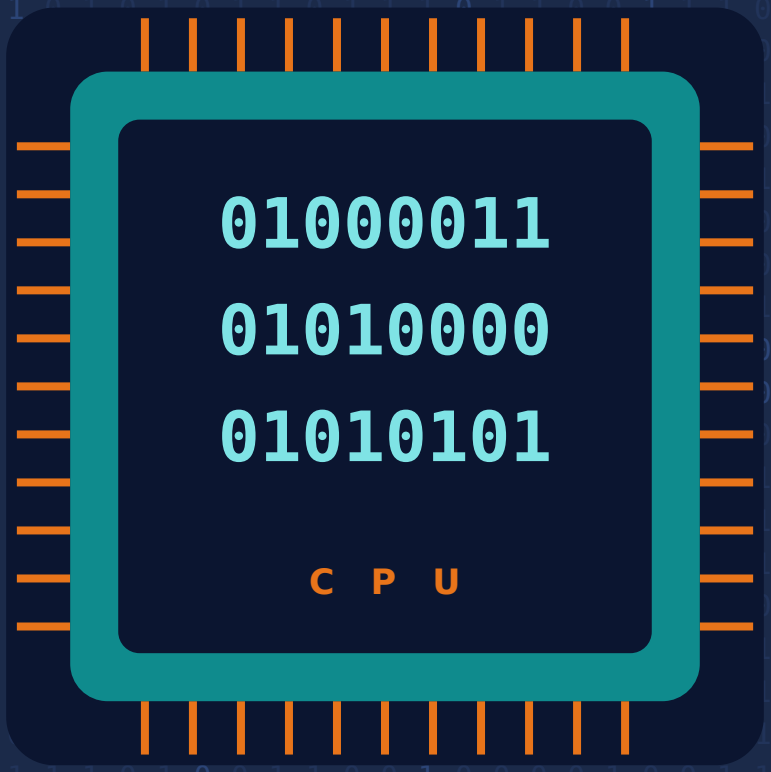
Made by Subhadeep Paul



Computer Systems & Data Representation

A Simple, Complete & Illustrated Study Book

Definitions • Examples • Diagrams • Step-by-step Solutions

Table of Contents

Chapter 1: Generations of Computer Systems 6

1.1 Before the Generations (Early Calculating Devices) 6
1.2 First Generation (1940 – 1956): Vacuum Tubes 6
1.3 Second Generation (1956 – 1963): Transistors 7
1.4 Third Generation (1964 – 1971): Integrated Circuits 7
1.5 Fourth Generation (1971 – Present): Microprocessors 8
1.6 Fifth Generation (Present and Beyond): Artificial Intelligence 8
1.7 Comparison of All Generations 8

Chapter 2: Definition of Computer 10

2.1 Data and Information 10
2.2 The IPO Cycle (Input – Process – Output) 10
2.3 Main Parts of a Computer 11
2.4 Stored Program Concept 12

Chapter 3: Characteristics of Computer 13

3.1 Limitations of Computers 15

Chapter 4: Types of Computers 16

4.1 Classification by Size and Power 16
4.2 Classification by Working Principle (Type of Data) 18
4.3 Classification by Purpose 18

Chapter 5: Usage of Computers in Various Fields 20

5.1 Education 20
5.2 Research 21
5.3 Business 21
5.4 Media House 21
5.5 Animation Industry 22
5.6 Space Technology 22

Chapter 6: Memory Units: Bit and Byte 24

6.1 Bit 24
6.2 Nibble 24
6.3 Byte 25

6.4 Bigger Memory Units . . . 25
6.5 Conversion Examples . . . 26
6.6 Memory vs Storage . . . 26

Chapter 7: Indian Knowledge System of Number Representation . . . 27

7.1 The Decimal Place-Value System . . . 27
7.2 Zero (Shunya) . . . 28
7.3 Indian Numerals → The Numbers We Use Today . . . 28
7.4 Names of Large Numbers and Indian Digit Grouping . . . 28
7.5 Pingala and the Binary Idea . . . 29
7.6 Katapayadi System (Letters as Numbers) . . . 30
7.7 Word Numerals (Bhuta-Sankhya) . . . 30
7.8 Great Indian Mathematicians . . . 30
7.9 Connection with Modern Computers . . . 31

Chapter 8: Number Systems . . . 32

8.1 Decimal Number System (Base 10) . . . 32
8.2 Binary Number System (Base 2) . . . 32
8.3 Octal Number System (Base 8) . . . 33
8.4 Hexadecimal Number System (Base 16) . . . 33
8.5 Conversions Between Number Systems . . . 34

Chapter 9: Character Representation . . . 37

9.1 Number of Characters and Bits . . . 37
9.2 Common Character Codes . . . 37

Chapter 10: 1's Complement, 2's Complement and N's Complement . . . 39

10.1 Two Types of Complement in Any Base . . . 39
10.2 Decimal Complements (9's and 10's) . . . 39
10.3 1's Complement (Binary) . . . 39
10.4 2's Complement (Binary) . . . 40
10.5 N's Complement in Other Bases (Octal and Hexadecimal) . . . 40
10.6 Representing Negative Numbers (Signed Numbers) . . . 40
10.7 Subtraction Using Complements (Decimal Idea) . . . 42

Chapter 11: Binary Arithmetic . . . 43

11.1 Binary Addition . . . 43
11.2 Binary Subtraction . . . 44

11.3 Binary Multiplication (Short Note) . . . 45
11.4 Binary Division . . . 45

Chapter 12: ASCII Code . . . 47

12.1 Structure of ASCII . . . 47
12.2 ASCII Table of Letters and Digits . . . 48
12.3 Worked Examples . . . 49
12.4 Extended ASCII and Limitations . . . 49

Chapter 13: Hardware and Software . . . 50

13.1 Hardware . . . 50
13.2 Software . . . 51
13.3 Hardware vs Software . . . 52

Chapter 14: Concept of Booting . . . 53

14.1 Why is Booting Needed? . . . 53
14.2 Types of Booting . . . 53
14.3 Steps of Booting . . . 54
14.4 BIOS vs UEFI . . . 56

Chapter 15: Language Processors . . . 57

15.1 Types of Programming Languages . . . 57
15.2 Language Processor (Translator) . . . 58
15.3 Assembler . . . 58
15.4 Compiler . . . 59
15.5 Interpreter . . . 59
15.6 Compiler vs Interpreter vs Assembler . . . 60

Chapter 16: Quick Reference Sheet . . . 61

16.1 Powers of 2 . . . 61
16.2 Number Systems at a Glance . . . 61
16.3 Complement Rules . . . 61
16.3 Important ASCII Values . . . 61
16.4 Boot Sequence in One Line . . . 61

Preface – How to Use This Book

This book covers the complete unit **“Computer Systems & Data Representation”** in simple language. Every topic is explained with an **exact definition**, easy **examples**, **diagrams** and step-by-step solved problems.

You will see coloured boxes throughout the book:

DEFINITION
Definition box – the exact definition. Learn these word for word for exams.

EXAMPLE
Example box – a simple example or solved problem to make the idea clear.

REMEMBER
Remember box – tips, shortcuts and important notes.

QUICK REVISION
✓ **Quick Revision box** – a short summary at the end of each chapter.

Topics Covered

Chapter	Topic
1	Generation of Computer Systems
2	Definition of Computer
3	Characteristics of Computer
4	Types of Computers
5	Usage of Computers: Education, Research, Business, Media House, Animation Industry, Space Technology
6	Memory Units – Bit, Byte
7	Indian Knowledge System of Number Representation
8	Number Systems – Decimal, Binary, Octal, Hexadecimal
9	Character Representation
10	1's Complement, 2's Complement, N's Complement
11	Binary Arithmetic – Addition, Subtraction, Division
12	ASCII
13	Hardware and Software
14	Concept of Booting – POST and Loading of OS
15	Language Processors – Assembler, Interpreter, Compiler
16	Quick Reference Sheet

NOTE
All diagrams in this book are drawn as vector graphics, so they stay sharp when you zoom in.

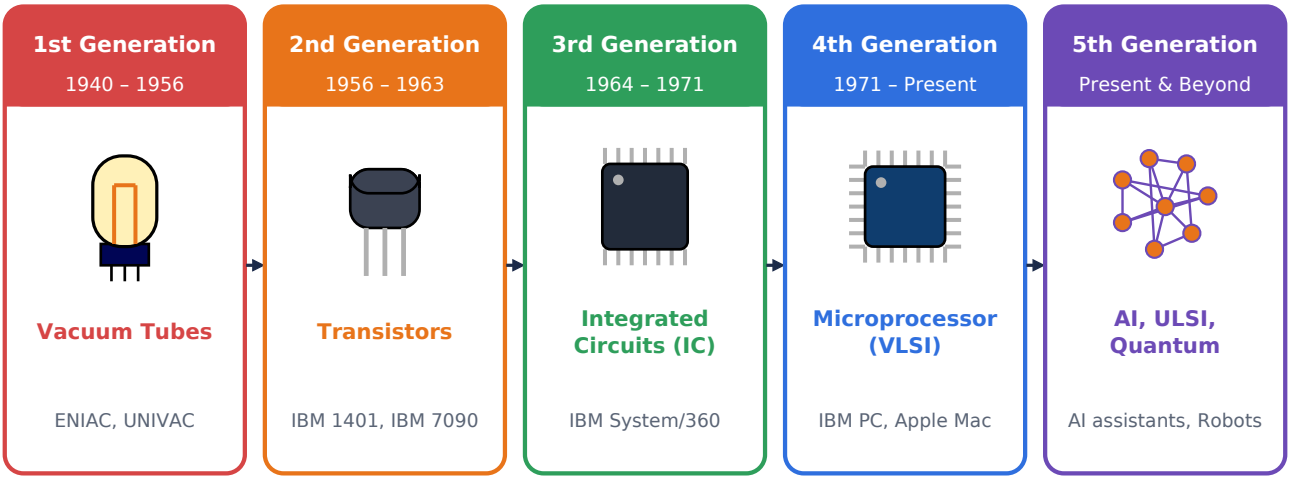
CHAPTER 1

Generations of Computer Systems

How computers changed from room-sized machines to smart devices

A **generation** of computers means a big change in the **technology** used to build them. Every time a new technology was invented, computers became **smaller, faster, cheaper and more reliable**. Scientists have divided this journey into **five generations**.

DEFINITION
Generation of computers: A stage in the development of computers that is identified by the main electronic technology (hardware) used to build them.



Smaller → Faster → Cheaper → More Reliable

Figure 1.1 – The five generations of computers at a glance

1.1 Before the Generations (Early Calculating Devices)

Before electronic computers, people used mechanical tools to calculate:

- **Abacus** (about 2400 BCE) – a frame with beads used to add, subtract, multiply and divide.
- **Pascaline** (1642) – a gear-based adding machine made by Blaise Pascal.
- **Analytical Engine** (1830s design) – designed by **Charles Babbage**, who is called the **Father of the Computer**. It had an input, a "mill" (like a CPU) and a "store" (like memory).
- **Ada Lovelace** wrote the first algorithm for this machine, so she is called the **first programmer**.

1.2 First Generation (1940 – 1956): Vacuum Tubes

DEFINITION
Vacuum tube: A glass tube with all the air removed, used as an electronic switch/amplifier. It controls the flow of electric current and can represent ON (1) and OFF (0).

The first generation computers were built with thousands of vacuum tubes.

Main features

- Used **vacuum tubes** for circuits and **magnetic drums** for memory.
- Very **big in size** – a single computer could fill a whole room.
- Produced a lot of **heat** and used huge amounts of **electricity**; tubes burnt out often.
- Programs were written in **machine language** (only 0s and 1s). Input was by punched cards and paper tape.
- Could solve only one problem at a time; very **slow and costly**.

EXAMPLE
ENIAC (1946) used about 18,000 vacuum tubes and weighed about 27 tonnes. **UNIVAC I** (1951) was the first commercially sold computer. Other examples: EDVAC, IBM 701.

1.3 Second Generation (1956 – 1963): Transistors

DEFINITION
Transistor: A tiny semiconductor device that works as a switch or amplifier. It was invented in 1947 at Bell Labs and replaced the vacuum tube. It is smaller, cooler, cheaper and does not burn out easily.

- Used **transistors** and **magnetic core memory**; later magnetic tapes and disks for storage.
- Smaller, faster, more reliable and used **less power** than first generation.
- **Assembly language** and the first high-level languages **FORTRAN** (1957) and **COBOL** (1959) were used.
- Used in business (payroll, inventory) and science. Output on printers.

EXAMPLE
IBM 1401, IBM 7090, CDC 1604. In India, early machines such as the HEC-2M were installed at the Indian Statistical Institute, Kolkata, in this period.

1.4 Third Generation (1964 – 1971): Integrated Circuits

DEFINITION
Integrated Circuit (IC): A small chip of silicon on which many transistors, resistors and capacitors are placed together to make a complete electronic circuit. It was developed by **Jack Kilby** and **Robert Noyce** (about 1958–59).

- Hundreds of transistors fit on **one chip** (SSI and MSI technology) – computers became much smaller.
- Users could work with a **keyboard** and **monitor** instead of punched cards.
- The **Operating System** appeared – it allowed many programs to run together (**multiprogramming**).
- High-level languages like BASIC became popular. Computers became **cheaper** and more reliable.

EXAMPLE
IBM System/360, PDP-8, ICL 1900 series.

1.5 Fourth Generation (1971 – Present): Microprocessors

DEFINITION
Microprocessor: A complete CPU built on a **single chip**. The first one was the **Intel 4004** (1971). This used **LSI** (Large Scale Integration) and then **VLSI** (Very Large Scale Integration) with millions of transistors on one chip.

- The **Personal Computer (PC)** was born – computers came to homes, schools and offices.
- Semiconductor memory (RAM), hard disks, floppy disks, later SSDs and pen drives.
- **GUI** (Graphical User Interface) with icons and **mouse** was introduced.
- Networking, **LAN** and the **Internet** grew. Languages: C, C++, Java, Python.
- Laptops, smartphones and tablets are all fourth-generation machines.

EXAMPLE
Altair 8800 (1975), Apple II (1977), IBM PC (1981), Apple Macintosh (1984), today's laptops and phones.

1.6 Fifth Generation (Present and Beyond): Artificial Intelligence

DEFINITION
Fifth generation computers are computers that use **ULSI** (Ultra Large Scale Integration), **parallel processing** and **Artificial Intelligence (AI)** so that machines can learn, reason and understand human language.

- Aim: computers that can **think, learn and take decisions** like humans.
- Features: voice recognition, natural-language understanding, robotics, machine learning, computer vision, expert systems.
- New technologies being developed: **quantum computing**, cloud and edge computing, nano-technology.

EXAMPLE
Smart voice assistants, AI chatbots, robots, face recognition, **self-driving car** systems and supercomputers like India's PARAM series.

1.7 Comparison of All Generations

Generation	Period	Technology	Language	Examples
First	1940–1956	Vacuum tubes	Machine language	ENIAC, UNIVAC
Second	1956–1963	Transistors	Assembly, FORTRAN, COBOL	IBM 1401, IBM 7090
Third	1964–1971	Integrated Circuits (SSI/MSI)	High-level (BASIC etc.)	IBM 360, PDP-8
Fourth	1971–Present	Microprocessor (LSI/VLSI)	C, C++, Java, Python	IBM PC, Apple Mac
Fifth	Present & Beyond	ULSI, AI, Quantum	AI & natural language	Robots, AI assistants

QUICK REVISION

✓ A generation = a change in the main **technology**.

✓ Gen 1: vacuum tube → Gen 2: transistor → Gen 3: IC → Gen 4: microprocessor → Gen 5: AI.

✓ Each generation: smaller size, higher speed, lower cost, better reliability.

✓ Charles Babbage = Father of the Computer; Ada Lovelace = first programmer.

CHAPTER 2

Definition of Computer

What a computer is and how it works

DEFINITION
A **computer** is an **electronic device** that accepts **data** as input, processes it using a set of **instructions (program)**, stores it when required, and gives the result as **information** (output). It works automatically and at very high speed.

The word *computer* comes from the Latin word **“computare”**, which means **“to calculate”**. Today a computer does much more than calculating – it plays videos, runs games, connects us to the world and controls machines.

2.1 Data and Information

DEFINITION
Data is a collection of raw facts, figures and symbols that have no meaning by themselves.

DEFINITION
Information is processed, organised and meaningful data that is useful for taking decisions.

EXAMPLE
Data: **45, 67, 89** (marks of three subjects). After processing (adding and dividing) we get **Total = 201** and **Average = 67**. This result is **information**: it tells the student how well he performed.

2.2 The IPO Cycle (Input – Process – Output)

Every computer works in a simple cycle:

- **Input** – data and instructions are given to the computer (typing, clicking, speaking, scanning).
- **Process** – the CPU works on the data as per the program.
- **Output** – the result is shown or printed.
- **Storage** – data and results can be saved for future use.

EXAMPLE
You type 5 + 3 on a calculator app (**input**) → the CPU adds them (**process**) → the screen shows 8 (**output**).

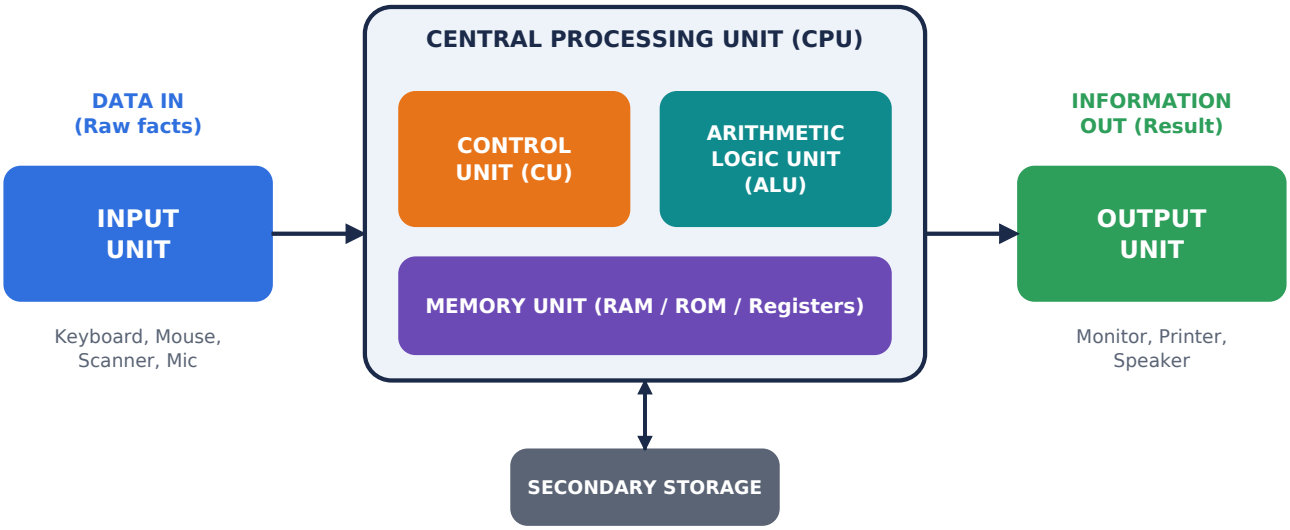


Figure 2.1 – Block diagram of a computer

2.3 Main Parts of a Computer

(a) Input Unit

Definition: The part of the computer that takes data and instructions from the user and converts them into a form (binary) that the computer can understand. *Examples:* keyboard, mouse, scanner, microphone, webcam.

(b) Central Processing Unit (CPU)

DEFINITION
The **CPU** is the “brain” of the computer. It executes instructions and controls all other parts. It contains the **Control Unit**, the **Arithmetic Logic Unit** and **Registers**.

- **Control Unit (CU):** Directs and coordinates all activities. It fetches an instruction, understands it and tells other parts what to do – like a traffic police.
- **Arithmetic Logic Unit (ALU):** Does all **arithmetic** operations (+, −, ×, ÷) and **logical** operations (comparisons like >, <, =, AND, OR, NOT).
- **Registers:** Very small, very fast storage places inside the CPU that hold the data being used right now.

(c) Memory Unit

Definition: The part that stores data, instructions and results. **Primary memory** (RAM, ROM) is directly used by the CPU. **Secondary storage** (hard disk, SSD, pen drive) keeps data permanently.

(d) Output Unit

Definition: The part that converts the computer’s binary results into a form humans can understand. *Examples:* monitor, printer, speaker, projector.

REMEMBER

CPU = CU + ALU (+ registers). The Control Unit *controls*, the ALU *calculates*, memory *remembers*.

2.4 Stored Program Concept

DEFINITION

Stored program concept (given by **John von Neumann**): both the **data** and the **program instructions** are stored in the computer's memory. The CPU fetches each instruction from memory, decodes it and executes it, one after another.

EXAMPLE

Instead of rewiring the machine for every new job (as in ENIAC), we now simply load a new program (for example a game or a calculator) into memory.

QUICK REVISION

✓ Computer = electronic device: Input → Process → Output (+ Storage).

✓ Data = raw facts; Information = processed, meaningful data.

✓ CPU has CU (controls), ALU (calculates) and registers (fast temporary storage).

✓ Stored program concept: program + data live in memory.

CHAPTER 3

Characteristics of Computer

Why computers are so powerful – and what they cannot do

DEFINITION

Characteristics of a computer are the special qualities and features that make a computer different from other machines and make it so useful for us.

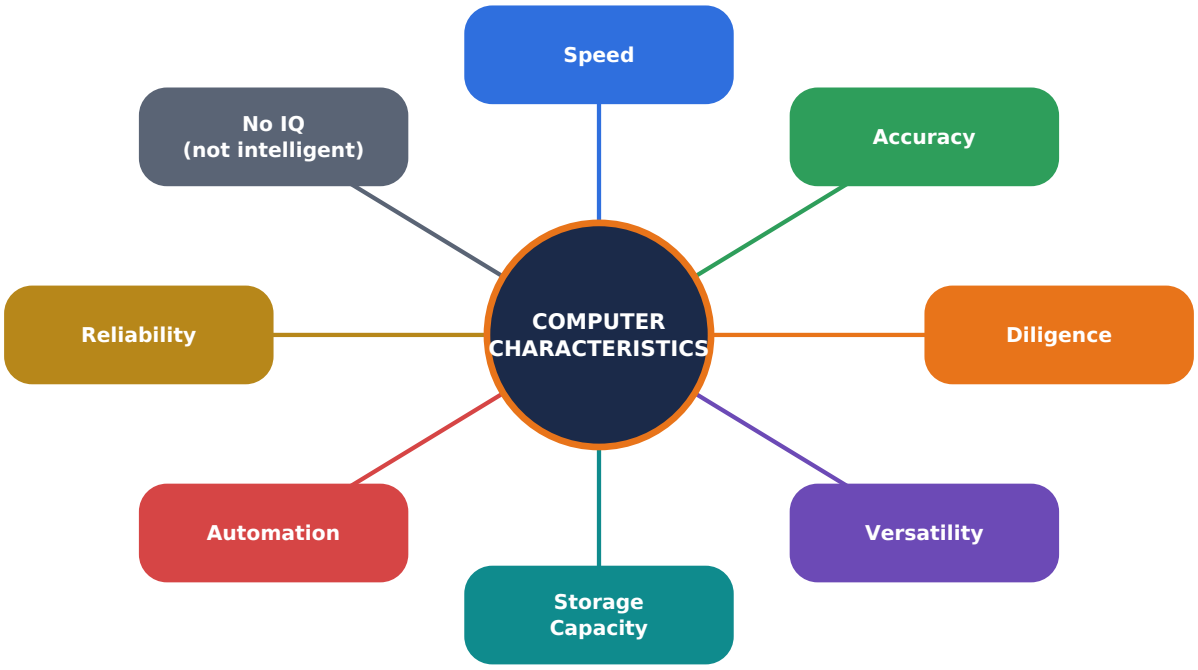


Figure 3.1 – Main characteristics of a computer

1. Speed

Definition: A computer can perform millions or billions of operations in **one second**. Speed is measured in MIPS (Million Instructions Per Second) or GHz.

EXAMPLE

A calculation that takes a person several hours (for example, adding a lakh numbers) is finished by a computer in a fraction of a second.

2. Accuracy

Definition: A computer gives **correct results** without making mistakes in calculation. Errors occur only if wrong data or a wrong program is given – this is called **GIGO** (Garbage In, Garbage Out).

EXAMPLE

If you enter marks as 95 instead of 59, the computer will faithfully calculate with 95 – the mistake was the input, not the machine.

3. Diligence

Definition: A computer **never gets tired, bored or careless**. It can work for hours with the same speed and accuracy.

EXAMPLE
A bank server processes millions of transactions 24×7 without taking a break.

4. Versatility

Definition: A computer can do **many different kinds of jobs** – only the program has to be changed.

EXAMPLE
The same laptop can type a letter, play music, edit a video, run games and join an online class.

5. Storage Capacity

Definition: A computer can **store huge amounts** of data and instructions and retrieve them in a moment.

EXAMPLE
A 1 TB hard disk can hold lakhs of photos or thousands of movies.

6. Automation

Definition: Once a program is loaded, a computer carries out the tasks **by itself** without human help at each step.

EXAMPLE
An ATM checks your card, asks the PIN, verifies it and gives cash automatically.

7. Reliability

Definition: Computers are **dependable**: they give the same output for the same input every time.

EXAMPLE
Railway reservation systems give consistent results to thousands of people.

8. No IQ (Not Intelligent)

Definition: A computer **cannot think or decide on its own**. It does only what it is told to do by a program.

EXAMPLE
A computer cannot cook a meal or decide what to do on a holiday unless somebody has programmed it.

9. No Feelings

Definition: A computer has **no emotions** or tastes. It does not feel happy, tired or sad.

EXAMPLE
It treats all data in the same way – it does not “like” or “dislike” a file.

10. Remembering Power / Memory

Definition: It can **remember data for a very long time** and never forgets unless told to delete.

EXAMPLE
Photos you saved years ago are still there, exactly as they were.

3.1 Limitations of Computers

- A computer **cannot think**; it needs a program for every task.
- It cannot take decisions on its own or learn without being programmed (until AI is added).
- It **depends on electricity** and hardware; if the power fails, work stops.
- It cannot correct wrong data by itself (GIGO).

QUICK REVISION
✓ Speed, accuracy, diligence, versatility, storage, automation, reliability are the strengths.
✓ GIGO = wrong input gives wrong output.
✓ A computer has no IQ and no feelings – it follows instructions.

CHAPTER 4

Types of Computers

Classified by size, working and purpose

Computers come in many shapes and sizes. They are classified in three common ways: by **size and power**, by **how they work with data**, and by **the purpose** they are made for.

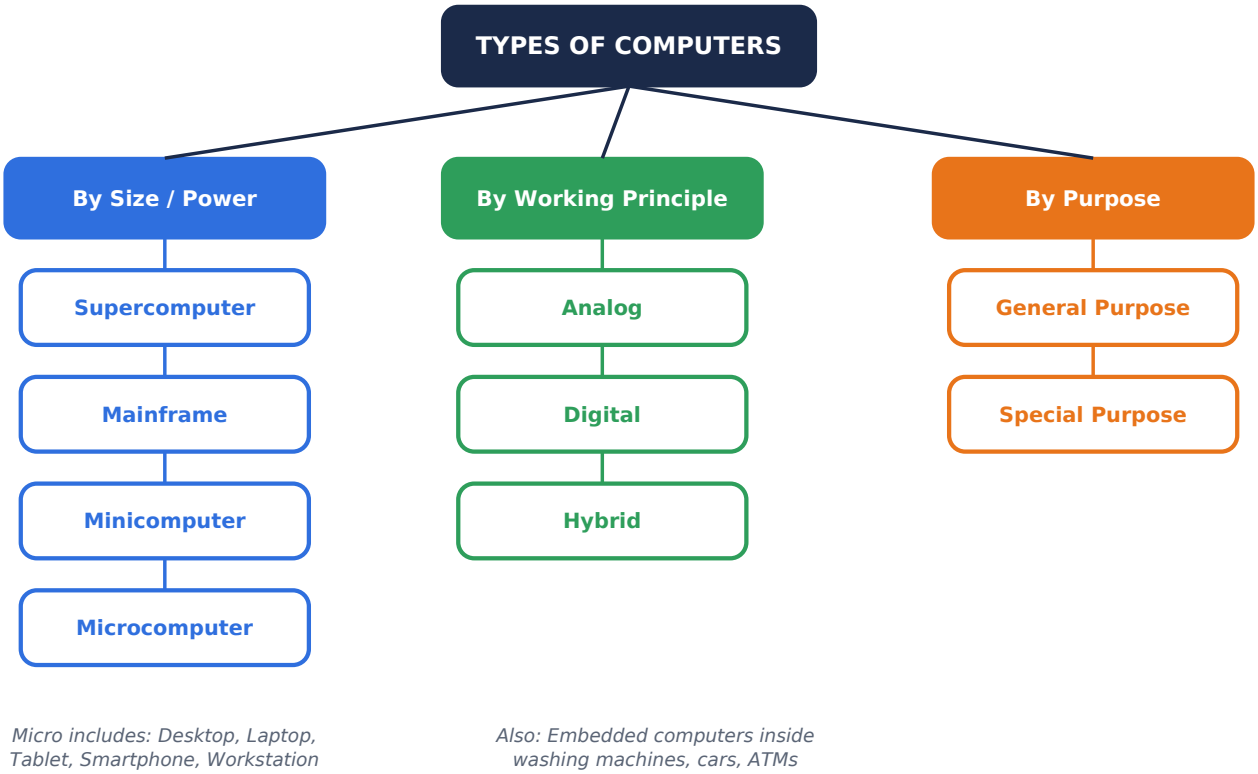


Figure 4.1 – Classification of computers

4.1 Classification by Size and Power

(a) Supercomputer

DEFINITION
A **supercomputer** is the **fastest and most powerful** computer. It has thousands of processors working in parallel and can do trillions of calculations per second (measured in FLOPS).

Used for: weather forecasting, space research, nuclear simulations, drug discovery, climate studies, artificial intelligence training.

EXAMPLE
PARAM series by C-DAC (India), **Fugaku** (Japan), **Frontier** (USA). India's National Supercomputing Mission uses PARAM machines at many institutes.

(b) Mainframe Computer

DEFINITION
A **mainframe** is a large, powerful computer that supports **hundreds or thousands of users at the same time** through terminals. It has huge storage and is very reliable.

Used for: banks, insurance, airlines, railways and government offices that handle millions of records.

EXAMPLE
IBM Z series; a bank's central system that handles all customer accounts.

(c) Minicomputer (Mid-range)

DEFINITION
A **minicomputer** is a medium-sized, multi-user computer, smaller and cheaper than a mainframe but more powerful than a PC. It supports tens or hundreds of users.

EXAMPLE
PDP-11 and VAX; used in small companies, labs and manufacturing plants.

(d) Microcomputer (Personal Computer)

DEFINITION
A **microcomputer** is a small computer whose CPU is a single **microprocessor**. It is mostly used by **one person** at a time (hence "personal").

Type	Description	Example
Desktop	Fixed on a table; separate monitor, keyboard, CPU box	Office / home PC
Laptop / Notebook	Portable, with built-in screen, keyboard and battery	Dell, HP, MacBook
Tablet	Touch-screen portable computer	iPad, Android tablet
Smartphone	Mobile phone with computer power and internet	Android phone, iPhone
Workstation	Powerful single-user PC for design, 3D and engineering	CAD / animation machine

(e) Embedded Computer

A tiny computer built **inside** another device to control it. *Examples:* washing machine, microwave oven, car engine control unit, smart TV, ATM, traffic lights.

Feature	Super	Mainframe	Mini	Micro
Speed	Fastest	Very fast	Fast	Moderate
Size	Very large (room)	Large	Medium	Small
Users	Few big jobs	Thousands	Tens–hundreds	One
Cost	Extremely high	Very high	High	Low
Use	Weather, space	Banks, airlines	Small firms	Home, office

4.2 Classification by Working Principle (Type of Data)

DEFINITION
Analog computer: Works on **continuous** physical quantities such as temperature, pressure, speed or voltage and shows the result as a measurement.

EXAMPLE
Speedometer of a car, mercury thermometer, analog clock, old analog voltmeters.

DEFINITION
Digital computer: Works on **discrete data** in the form of 0s and 1s (binary digits). It counts instead of measuring. It is accurate and the most common type today.

EXAMPLE
Desktop PC, laptop, smartphone, digital watch, calculator.

DEFINITION
Hybrid computer: Combines the features of analog and digital computers – it measures with the analog part and processes with the digital part.

EXAMPLE
ICU patient monitors (measure heartbeat, convert to digital numbers), ECG machines, petrol pump meters, flight simulators.

Analog	Digital	Hybrid
Continuous data	Discrete (0 and 1)	Both
Less accurate	Highly accurate	Accurate
Measures	Counts	Measures + counts
Speedometer	PC / Laptop	ICU monitor

4.3 Classification by Purpose

DEFINITION
General purpose computer: Designed to do **many different tasks** by running different programs.

EXAMPLE
PC, laptop, smartphone.

DEFINITION
Special purpose computer: Designed to do **one specific task** only. The program is permanent and cannot be changed easily.

EXAMPLE

Computer in a traffic signal, ATM, washing machine, a missile guidance system, a digital watch.

QUICK REVISION

✓ By size: Super → Mainframe → Mini → Micro (power decreases).

✓ By data: Analog (measures), Digital (counts), Hybrid (both).

✓ By purpose: General purpose (many jobs), Special purpose (one job).

✓ Microcomputers: desktop, laptop, tablet, smartphone, workstation.

CHAPTER 5

Usage of Computers in Various Fields

Education, Research, Business, Media, Animation and Space

Computers are used almost everywhere today. In this chapter we study six important fields.

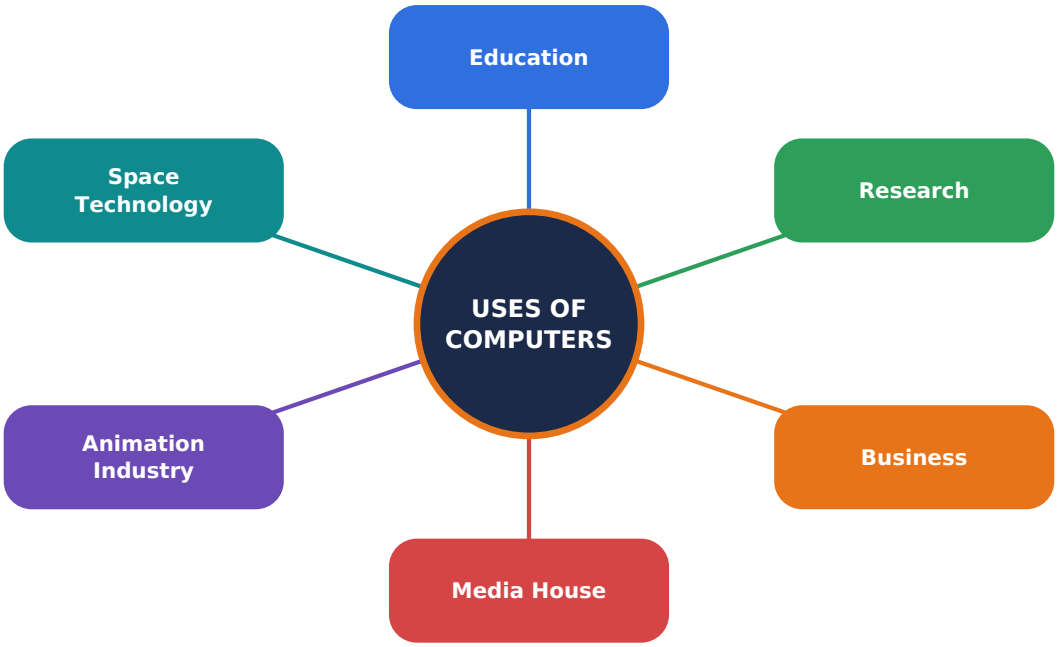


Figure 5.1 – Fields where computers are widely used

5.1 Education

DEFINITION
Computer in education means using computers and digital tools for teaching, learning, administration and examinations.

- **Smart classrooms:** projectors and digital boards show animations and videos.
- **E-learning / online classes:** students learn from anywhere using the internet.
- **Digital libraries and e-books:** thousands of books can be searched in seconds.
- **Online exams and results:** automatic checking of MCQs and fast result processing.
- **Simulations:** students can do virtual science experiments safely.
- **School administration:** admissions, attendance, fee records.

EXAMPLE
SWAYAM, **DIKSHA**, **NPTEL** (Indian platforms for free online courses), Google Classroom, online entrance examinations.

5.2 Research

DEFINITION
Computer in research means using computers to collect, store, analyse and model data and to perform experiments that are too difficult, costly or dangerous in real life.

- Fast **analysis of large data** (statistics, graphs).
- **Simulation and modelling** – weather, earthquakes, medicines.
- Searching research papers and sharing results through the internet.
- Controlling and recording laboratory instruments.

EXAMPLE
Climate-change models, **genome sequencing**, drug discovery for new vaccines, particle experiments at CERN that produce huge data analysed by computers.

5.3 Business

DEFINITION
Computer in business means using computers to run the daily work of a company such as accounting, sales, inventory, banking and communication.

- **Accounting and billing** (Tally, spreadsheets, GST returns).
- **Banking:** ATMs, net banking, UPI payments, mobile banking.
- **E-commerce:** online shopping, digital marketing, online ticket booking.
- **Inventory and payroll** management; **ERP** and **CRM** software.
- **Stock market** trading and data analysis; video meetings and e-mail.

EXAMPLE
Amazon/Flipkart online shopping, UPI payment using a phone, railway ticket booking on IRCTC.

5.4 Media House

DEFINITION
Media house means newspaper, TV, radio, film and online news companies. Computers help them create, edit, publish and broadcast content.

- **Desktop publishing (DTP):** designing newspapers and magazines (Adobe InDesign, Photoshop).
- **Video and audio editing:** editing news, adding graphics and music (Premiere Pro, DaVinci Resolve).
- **Live broadcasting and streaming:** TV channels, YouTube Live, podcasts.
- **News gathering:** reporters send stories and photos through the internet instantly.
- **Social media and digital marketing:** analytics to know what the audience likes.

EXAMPLE
A news channel shows live scores, 3D weather maps and breaking news graphics – all created on computers.

5.5 Animation Industry

DEFINITION
Animation is the art of making still pictures appear to move. **Computer animation** creates these moving pictures using software – in 2D or 3D.

- **Modelling:** making a 3D shape of a character.
- **Rigging:** adding a skeleton so the model can move.
- **Animation:** giving movement frame by frame.
- **Rendering:** the computer calculates light, colour and shadow to make final images (needs powerful PCs).
- **VFX and motion capture:** special effects and recording real actors' movements.

EXAMPLE
Toy Story (1995) was the first full-length computer-animated film. Software: Blender, Autodesk Maya, Adobe After Effects. Indian examples: *Chhota Bheem* and VFX in films like *Baahubali*.

5.6 Space Technology

DEFINITION
Computer in space technology means using computers in designing, launching, controlling and tracking rockets, satellites and space missions.

- **Calculating the path (trajectory)** of rockets and satellites.
- **On-board computers** that control spacecraft and rovers by themselves.
- **Ground stations** that track satellites and receive data.
- **Image processing:** converting satellite signals into pictures of Earth, Moon and Mars.
- **Simulations** to test missions before launch; autonomous **navigation using cameras** (computer vision).

EXAMPLE
ISRO: **Mangalyaan** (Mars Orbiter Mission) entered Mars orbit in 2014. **Chandrayaan-3** made a soft landing near the Moon's south pole on **23 August 2023**, guided by its on-board computers.

QUICK REVISION

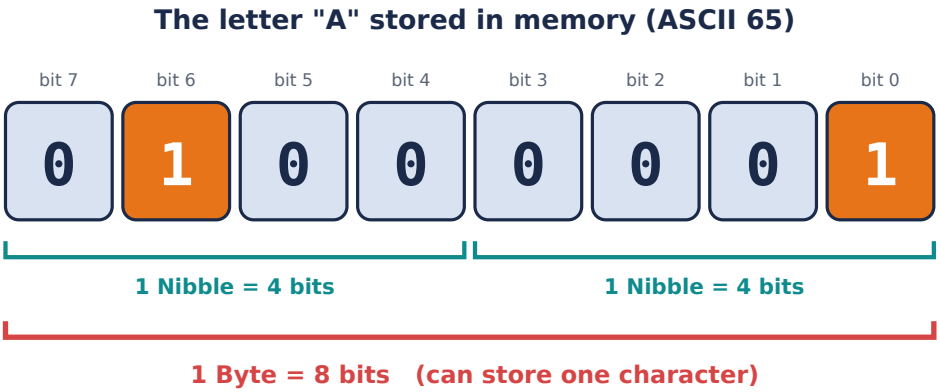
- ✓ Education: smart class, e-learning, online exams.
- ✓ Research: analysis, simulation, modelling.
- ✓ Business: accounting, banking, e-commerce, ERP.
- ✓ Media: DTP, video editing, live broadcast.
- ✓ Animation: model → rig → animate → render.
- ✓ Space: trajectory, on-board control, image processing.

CHAPTER 6

Memory Units: Bit and Byte

How the size of data and memory is measured

A computer stores everything – text, photos, songs, videos – as a long series of **0s and 1s**. To measure how much data is stored, we use **memory units**.



Each box is one BIT (Binary digit): either 0 (OFF) or 1 (ON)

Figure 6.1 – A bit, a nibble and a byte

6.1 Bit

DEFINITION

A **bit** (short for **Binary digIT**) is the **smallest unit of data** in a computer. It can have only one of two values: **0** or **1**.

A bit is like a light switch: **ON = 1** and **OFF = 0**. Inside the computer, these are represented by electric signals (high voltage and low voltage).

EXAMPLE

1 bit can show 2 things (0 or 1). 2 bits can show 4 things (00, 01, 10, 11). 3 bits can show 8 things. **n bits can show 2^n different values.**

6.2 Nibble

DEFINITION

A **nibble** is a group of **4 bits**. One nibble can show $2^4 = 16$ values (0 to 15) – which is exactly one hexadecimal digit.

EXAMPLE

1010 is a nibble. It equals 10 in decimal and A in hexadecimal.

6.3 Byte

DEFINITION

A **byte** is a group of **8 bits**. It is the basic unit used to store **one character** (a letter, digit or symbol).

EXAMPLE

The letter **A** is stored as the byte **01000001**. The word “CAT” needs 3 bytes. One byte can show 2^8 = **256** different values (0 to 255).

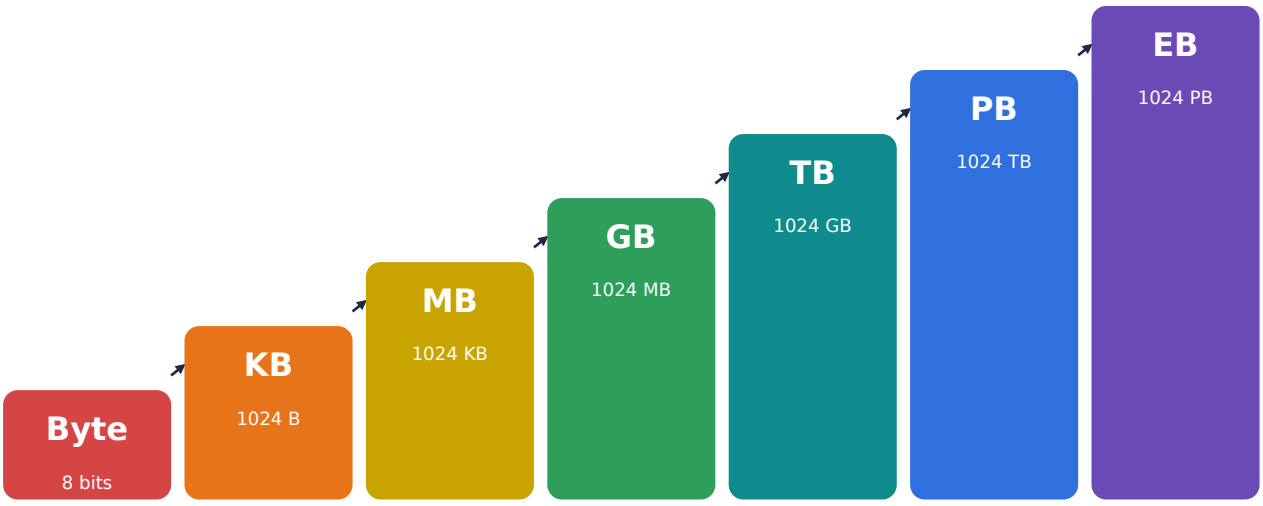
Word

DEFINITION

A **word** is a group of bits that the CPU processes **together at one time**. Its size (the word length) may be 16, 32 or 64 bits. A “64-bit computer” handles 64 bits at once.

6.4 Bigger Memory Units

Since a byte is very small, bigger units are used. Each unit is **1024 times** the previous one (because 1024 = 2^{10}, a power of 2).



Each step is × 1024 (that is 2^{10}) bigger than the one before

Figure 6.2 – Memory units grow by 1024 times at each step

Unit	Short form	Equal to	Approx. example
Bit	b	Smallest unit (0 or 1)	One ON/OFF switch
Nibble	–	4 bits	One hex digit
Byte	B	8 bits	One character
Kilobyte	KB	1024 Bytes	A short text page
Megabyte	MB	1024 KB	A photo / a song
Gigabyte	GB	1024 MB	A movie / mobile storage
Terabyte	TB	1024 GB	A laptop hard disk

Unit	Short form	Equal to	Approx. example
Petabyte	PB	1024 TB	Data of a big internet company
Exabyte	EB	1024 PB	Global internet traffic scale

REMEMBER

Storage companies often use 1 KB = 1000 bytes (decimal) while computers use 1024. That is why a “500 GB” disk shows slightly less capacity in your computer. The exact binary units are written KiB, MiB, GiB.

6.5 Conversion Examples

WORKED EXAMPLES

Q1. Convert 3 KB into bytes.
3 × 1024 = **3072 bytes**.

Q2. Convert 5 GB into MB.
5 × 1024 = **5120 MB**.

Q3. Convert 4096 MB into GB.
4096 ÷ 1024 = **4 GB**.

Q4. How many bits are in 2 bytes?
2 × 8 = **16 bits**.

Q5. How many bits are in 1 KB?
1024 × 8 = **8192 bits**.

6.6 Memory vs Storage

Primary Memory (Main memory)	Secondary Storage
Directly used by the CPU	Used to keep data permanently
Fast but small; RAM loses data when power is off	Slower but very large; keeps data without power
RAM, ROM, Cache	Hard disk, SSD, pen drive, DVD, memory card

QUICK REVISION

✓ 1 byte = 8 bits; 1 nibble = 4 bits; n bits → 2^n values.

✓ 1 KB = 1024 B; 1 MB = 1024 KB; 1 GB = 1024 MB; 1 TB = 1024 GB; 1 PB = 1024 TB.

✓ Bigger unit → smaller unit: multiply by 1024. Smaller → bigger: divide by 1024.

CHAPTER 7

Indian Knowledge System of Number Representation

Zero, place value, binary ideas and more from ancient India

DEFINITION

The **Indian Knowledge System (IKS)** is the large and continuous body of knowledge developed in India over thousands of years in subjects such as mathematics, astronomy, medicine, language, music and philosophy. It was passed on through texts, teachers and traditions.

In this chapter we look at the part of IKS which is closely connected to computers – **how numbers were represented**. India gave the world the **decimal place-value system** and **zero**, which are the base of all modern mathematics and computing. Ancient Indian scholars also used **binary-like ideas** long before computers existed.

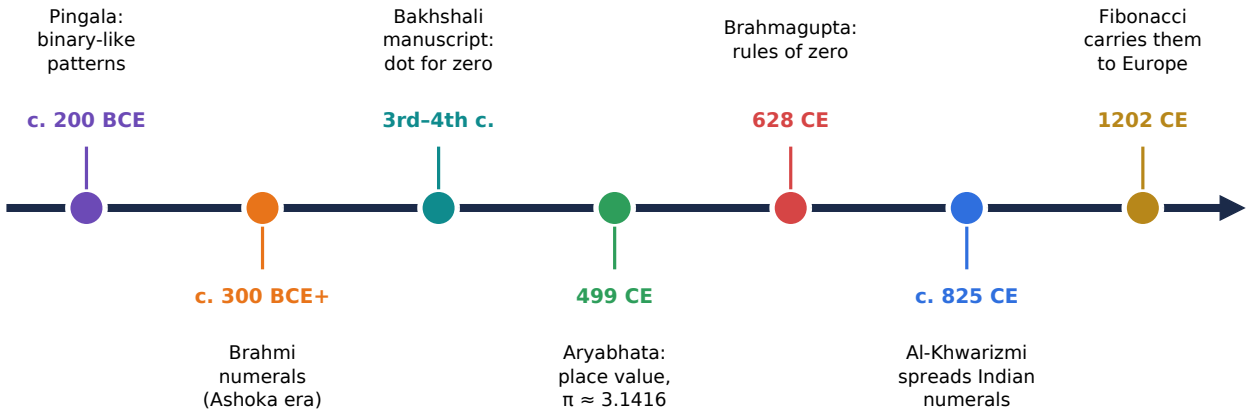


Figure 7.1 – Timeline of Indian number ideas and how they travelled to the world

7.1 The Decimal Place-Value System

DEFINITION

A **place-value system** is a method of writing numbers in which the **value of a digit depends on its position**. In the Indian decimal system there are **ten symbols (0–9)** and each place is **ten times** the place on its right.

EXAMPLE

In 505, the first 5 is worth 5 × 100 = 500 and the last 5 is worth 5 × 1 = 5. The same digit gives different values because of its position. In 50 and 5 the zero acts as a **placeholder**, so we can tell 5, 50 and 500 apart.

Ancient Indian texts, including the works of **Aryabhata** (499 CE), clearly describe the ideas of place value and name the places as powers of ten. This is far simpler than systems like Roman numerals (I, V, X, L, C) where there is no zero and calculation is difficult.

7.2 Zero (Shunya)

DEFINITION
Zero (shunya) is a number that stands for “nothing” or “empty”, and also a symbol used as a **placeholder** in place-value numbers.

- A **dot** used as zero is seen in the **Bakhshali manuscript**, one of the oldest Indian mathematical manuscripts (radiocarbon dating puts parts of it as early as the 3rd–4th century CE).
- **Brahmagupta** (628 CE), in *Brahmasphutasiddhanta*, gave rules for using zero as a number: $a + 0 = a$, $a - a = 0$ and $a \times 0 = 0$.
- The Sanskrit word **shunya** became *sifr* in Arabic, then *zephirum* in Latin, and finally **“zero”** and **“cipher”** in English.

REMEMBER
Without zero, we could not write numbers like 10, 100 or 1000 clearly – and binary numbers such as 1000 or 10110 would be impossible. Computers depend on 0 and 1!

7.3 Indian Numerals → The Numbers We Use Today

The numerals used in ancient India (**Brahmi numerals**, seen in the inscriptions of Emperor Ashoka) slowly developed into the **Devanagari numerals** and into the **Hindu–Arabic numerals (0 to 9)** that the whole world uses. Arab scholars such as **Al-Khwarizmi** (c. 825 CE) wrote about them, and **Fibonacci** (1202 CE) introduced them in Europe in his book *Liber Abaci*.

Number	0	1	2	3	4	5	6	7	8	9
Devanagari	०	१	२	३	४	५	६	७	८	९
Sanskrit name	shunya	eka	dvi	tri	chatur	pancha	shat	sapta	ashta	nava

7.4 Names of Large Numbers and Indian Digit Grouping

Ancient Indian texts gave a **special name for every power of ten**, which made it easy to speak and write very large numbers. Some of the names still used in India today:

Number	Name	Value
1	Eka	1
10	Dasha	10^1
100	Shata	10^2
1,000	Sahasra	10^3
10,000	Ayuta	10^4
1,00,000	Laksha (Lakh)	10^5
1,00,00,000	Koti (Crore)	10^7

Indian vs International digit grouping

DEFINITION
In the **Indian system**, the **last three digits** are grouped first (thousands) and then every **two digits** (lakhs, crores). In the **International system** every **three digits** are grouped (thousand, million, billion).

EXAMPLE
The number 12345678 is written **1,23,45,678** in the Indian system (1 crore 23 lakh 45 thousand 678) and **12,345,678** in the International system (12 million ...). Also: **1 crore = 100 lakh = 10 million**.

7.5 Pingala and the Binary Idea

DEFINITION
Pingala (around 200 BCE) wrote *Chandahshastra*, the Sanskrit book on **poetic metres**. He described every verse pattern as a sequence of two kinds of syllables: **laghu** (short, 1 beat) and **guru** (long, 2 beats). This is the same idea as a **binary number** that uses only two symbols.

If we write laghu = 0 and guru = 1, then every pattern of syllables becomes a binary number. For 3 syllables there are 2^3 = **8** possible patterns – exactly the numbers 000 to 111.

Pingala's idea: short (laghu) and long (guru) syllables work like binary 0 and 1

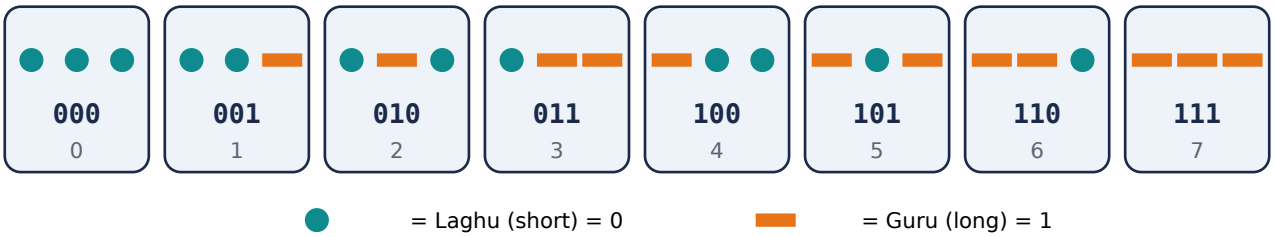


Figure 7.2 – All patterns of three syllables as binary numbers

- Pingala gave rules to **list all patterns** and to find the **position number** of any pattern – a method similar to converting between binary and decimal.
- He also described a number triangle called **Meru Prastara**, known in the West as **Pascal's Triangle**.

REMEMBER
Modern computers use two states (ON/OFF) and binary numbers. Pingala's idea is an early example of representing information with just **two symbols**. (He did not invent the computer, of course – but his idea is closely related to binary coding.)

7.6 Katapayadi System (Letters as Numbers)

DEFINITION
The **Katapayadi system** is an ancient Indian method of **writing numbers using Sanskrit letters**. Each consonant stands for a digit; vowels have no value. It was used to put numbers inside verses and names so that they are easy to remember.

Digit	1	2	3	4	5	6	7	8	9	0
Letters	ka, ṭa, pa, ya	kha, ṭha, pha, ra	ga, ḍa, ba, la	gha, ḍha, bha, va	ṅa, ṇa, ma, śa	ca, ta, ṣa	cha, tha, sa	ja, da, ha	jha, dha	ña, na

Rule: numbers are read from **right to left** (*aṅkānāṃ vāmato gatiḥ*).

EXAMPLE
The Carnatic raga **Dheerasankarabharanam**: dha = 9, ra = 2 → “92” read right to left = **29** (it is the 29th melakarta raga). **Mayamalavagowla**: ma = 5, ya = 1 → “51” → **15** (15th melakarta).

7.7 Word Numerals (Bhuta-Sankhya)

DEFINITION
In **word numerals**, a number is written using **words for things that occur in a fixed count** in nature or tradition. It was popular in Sanskrit verse and astronomy texts.

Word	Chandra (Moon)	Netra (Eyes)	Agni (Fires)	Veda	Bana (Arrows)	Rishi (Sages)	Kha (Sky)
Value	1	2	3	4	5	7	0

EXAMPLE
“Veda – Netra” written as words stands for digits 4, 2; read right to left gives **24**.

7.8 Great Indian Mathematicians

Name	Time	Contribution
Pingala	c. 200 BCE	Binary-like patterns of syllables; Meru Prastara
Aryabhata	499 CE	Aryabhatiya – place value, value of π ≈ 3.1416, algebra, astronomy
Brahmagupta	628 CE	Rules of zero and negative numbers
Mahavira	9th century	Methods for LCM, fractions, permutations
Bhaskaracharya II	12th century	Lilavati and Bijaganita – arithmetic and algebra
Madhava of Sangamagrama	14th century	Infinite series for π and trigonometric functions

7.9 Connection with Modern Computers

Ancient Indian idea	Use in computers today
Decimal place-value system	Basis of all number systems (decimal, binary, octal, hex)
Zero as a number and placeholder	The digit 0 in binary; empty and null values
Pingala’s two-symbol patterns	Binary coding of data and instructions
Katapayadi / word numerals	Early forms of encoding numbers into symbols (like codes)

QUICK REVISION

✓ IKS gave the world the **decimal place-value system and zero**.

✓ Brahmagupta (628 CE) gave rules for zero; Aryabhata (499 CE) described place value.

✓ Pingala (c. 200 BCE) used short/long syllables like 0/1 – a binary idea.

✓ Katapayadi: letters represent digits, read from right to left.

✓ Indian grouping: 1,23,45,678; International: 12,345,678.

CHAPTER 8

Number Systems

Decimal, Binary, Octal and Hexadecimal with conversions

DEFINITION
A **number system** is a method of representing numbers using a fixed set of **digits (symbols)** and rules. The total number of digits used in a system is called its **base** or **radix**.

There are two types: **Positional** number systems (value depends on position – decimal, binary, octal, hexadecimal) and **non-positional** systems (like Roman numerals, where X always means 10).

System	Base	Digits used	Short name
Decimal	10	0 1 2 3 4 5 6 7 8 9	DEC
Binary	2	0 1	BIN
Octal	8	0 1 2 3 4 5 6 7	OCT
Hexadecimal	16	0–9 and A B C D E F	HEX

8.1 Decimal Number System (Base 10)

DEFINITION
The **decimal system** has **10 digits (0–9)**. Each position has a weight that is a **power of 10**. It is the system used in daily life.

EXAMPLE
3425 = $3\times10^3 + 4\times10^2 + 2\times10^1 + 5\times10^0 = 3000 + 400 + 20 + 5$.

8.2 Binary Number System (Base 2)

DEFINITION
The **binary system** has only **two digits: 0 and 1**. Each position has a weight that is a **power of 2** (..., 16, 8, 4, 2, 1). It is the language of computers.

Why do computers use binary? Electronic circuits have two easy-to-detect states – **ON/OFF** or **high/low voltage**. Two states are simple, cheap, fast and very reliable.

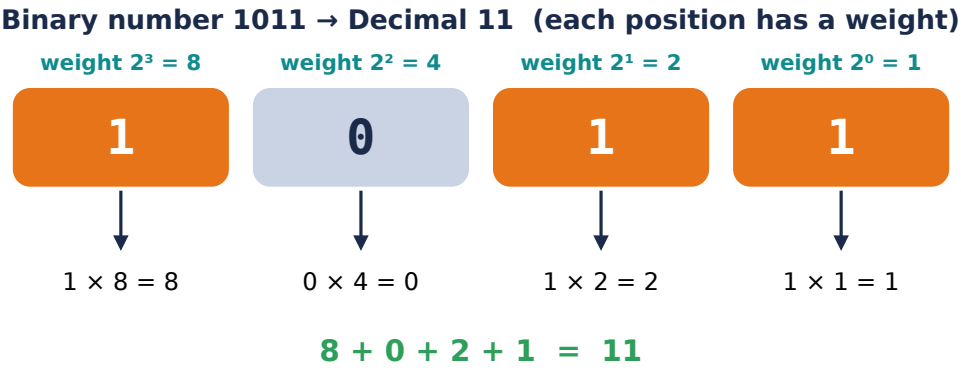


Figure 8.1 – Place values (weights) in binary

EXAMPLE
1011_2 = 1×8 + 0×4 + 1×2 + 1×1 = **11** in decimal.

8.3 Octal Number System (Base 8)

DEFINITION
The **octal system** has **8 digits (0–7)**. Each position is a **power of 8**. **One octal digit = 3 binary bits**.

Octal is a short way to write long binary numbers. It is used in Unix/Linux file permissions.

EXAMPLE
chmod 755: 7 = 111, 5 = 101, 5 = 101 → 111 101 101 = rwx r-x r-x. Also 725_8 = 7×64 + 2×8 + 5×1 = **469** in decimal.

8.4 Hexadecimal Number System (Base 16)

DEFINITION
The **hexadecimal system** has **16 symbols: 0–9 and A–F** where A = 10, B = 11, C = 12, D = 13, E = 14, F = 15. Each position is a **power of 16**. **One hex digit = 4 binary bits**.

Hexadecimal is used for **memory addresses**, **colour codes** in web pages and **MAC addresses**.

EXAMPLE
The colour **#FF0000** is red: FF = 255 red, 00 green, 00 blue. And $2F_{16}$ = 2×16 + 15 = **47**.

Equivalent values (0 to 15)

Decimal	Binary	Octal	Hex
0	0000	0	0
1	0001	1	1
2	0010	2	2
3	0011	3	3
4	0100	4	4
5	0101	5	5
6	0110	6	6
7	0111	7	7

Decimal	Binary	Octal	Hex
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

8.5 Conversions Between Number Systems

(A) Any base to Decimal – "Expanded form" method

Rule: Multiply each digit by its position weight (power of the base) and add all.

EXAMPLES

(i) $101101_2 = 1\times32 + 0\times16 + 1\times8 + 1\times4 + 0\times2 + 1\times1 = 32 + 8 + 4 + 1 = \mathbf{45}$

(ii) $725_8 = 7\times64 + 2\times8 + 5\times1 = 448 + 16 + 5 = \mathbf{469}$

(iii) $1A3_{16} = 1\times256 + 10\times16 + 3\times1 = 256 + 160 + 3 = \mathbf{419}$

(iv) $1011.101_2 = (8 + 2 + 1) + (1\times½ + 0\times¼ + 1\times⅛) = 11 + 0.5 + 0.125 = \mathbf{11.625}$

(B) Decimal to any base – "Repeated division" method

Rule: Divide the number by the base again and again. Write the remainder each time. Stop when the quotient is 0. Read the remainders from **bottom to top**.

Example 1: Convert 45_{10} to binary.

Division	Quotient	Remainder
45 ÷ 2	22	1
22 ÷ 2	11	0
11 ÷ 2	5	1
5 ÷ 2	2	1
2 ÷ 2	1	0
1 ÷ 2	0	1

Reading the remainders from bottom to top: $\mathbf{45_{10} = 101101_2}$.

Example 2: Convert 156_{10} to octal.

Division	Quotient	Remainder
156 ÷ 8	19	4
19 ÷ 8	2	3
2 ÷ 8	0	2

Bottom to top: **$156_{10} = 234_8$**.

Example 3: Convert 1000_{10} to hexadecimal.

Division	Quotient	Remainder
1000 ÷ 16	62	8
62 ÷ 16	3	14 = E
3 ÷ 16	0	3

Bottom to top: **$1000_{10} = 3E8_{16}$** (14 is written as E).

Decimal fraction to binary – "Repeated multiplication" method

Rule: Multiply the fraction by 2. The whole-number part (0 or 1) is the next binary digit. Repeat with the remaining fraction. Read digits from **top to bottom**.

EXAMPLE
Convert **0.625** to binary:
0.625 × 2 = 1.25 → **1**
0.25 × 2 = 0.50 → **0**
0.50 × 2 = 1.00 → **1**
So 0.625_{10} = **0.101_2**.

(C) Binary ↔ Octal (group of 3 bits)

Binary → Octal: Make groups of **3 bits** starting from the right (add zeros on the left if needed). Write the octal digit for each group.
Octal → Binary: Replace each octal digit by its 3-bit binary.

EXAMPLE
Binary 1101011 → Octal
Groups: **001** | **101** | **011** → 1 | 5 | 3 → **153_8**
Octal 345 → Binary
3 = 011, 4 = 100, 5 = 101 → **011100101_2**

(D) Binary ↔ Hexadecimal (group of 4 bits)

Binary → Hex: Make groups of **4 bits** from the right. **Hex → Binary:** Replace each hex digit by its 4-bit binary.

EXAMPLE
Binary 1101011 → Hex
Groups: **0110** | **1011** → 6 | B → **$6B_{16}$**
Hex 3A7 → Binary
3 = 0011, A = 1010, 7 = 0111 → **001110100111_2**

(E) Octal ↔ Hexadecimal (go through Binary)

EXAMPLE

Octal 345 → Hex

345_8 = 011 100 101 = 0 1110 0101 → **$E5_{16}$** (check: 3×64 + 4×8 + 5 = 229 = E5)

Hex 2B → Octal

$2B_{16}$ = 0010 1011 = 000 101 011 → **53_8** (check: 32 + 11 = 43 = 5×8 + 3)

REMEMBER

Shortcut: group bits in **3s for octal** and in **4s for hexadecimal**.

QUICK REVISION

- ✓ Base = number of digits in a system: Decimal 10, Binary 2, Octal 8, Hex 16.
- ✓ To decimal: multiply by weights and add. From decimal: repeated division, read remainders bottom to top.
- ✓ Binary → Octal: groups of 3; Binary → Hex: groups of 4.
- ✓ Hex letters: A=10, B=11, C=12, D=13, E=14, F=15.

CHAPTER 9

Character Representation

How letters, digits and symbols are stored as 0s and 1s

DEFINITION
Character representation is the method of converting **characters** (letters, digits, punctuation marks and symbols) into **binary codes** so that a computer can store, process and display them.

A computer understands only 0s and 1s. So every character is given a unique number, and that number is stored in binary. This table of characters and their numbers is called a **character set** or **coding scheme**.

DEFINITION
Character set / Character code: A list of characters in which every character has a unique numeric code.

EXAMPLE
When you press the key **A**, the keyboard sends the code 65. The computer stores it as **01000001** and later shows the shape “A” on screen.

9.1 Number of Characters and Bits

If a code uses **n bits**, it can represent 2^n different characters.

Code bits	Different characters	Used in
6 bits	$2^6 = 64$	BCD (old)
7 bits	$2^7 = 128$	ASCII
8 bits	$2^8 = 256$	Extended ASCII, EBCDIC
16–32 bits (variable)	Over 1.1 million code positions	Unicode (UTF-8, UTF-16, UTF-32)

9.2 Common Character Codes

(a) BCD – Binary Coded Decimal

DEFINITION
BCD represents each **decimal digit** separately by its own **4-bit** binary code.

EXAMPLE
The decimal number **25** in BCD: 2 → 0010 and 5 → 0101, so 25 = **0010 0101**. (In pure binary, 25 = 11001.)

(b) EBCDIC

DEFINITION

EBCDIC (Extended Binary Coded Decimal Interchange Code) is an **8-bit** code developed by **IBM**, used mainly in IBM mainframe computers. It can represent 256 characters.

(c) ASCII

DEFINITION

ASCII (American Standard Code for Information Interchange) is a **7-bit** code for 128 characters and is the most widely used simple code (studied in detail in Chapter 12).

(d) Unicode

DEFINITION

Unicode is a universal character set that gives a unique code to **every character of every language** in the world, including Hindi, Tamil, Bengali, Chinese, Arabic, and even emojis.

EXAMPLE

The Hindi letter □ has Unicode value **U+0905**. The English letter A is U+0041 (same as ASCII 65). The first 128 Unicode characters are identical to ASCII.

UTF-8 is the most popular way of storing Unicode: it uses 1 byte for English letters and up to 4 bytes for other characters. Most websites use UTF-8.

Feature	ASCII	EBCDIC	Unicode
Bits	7 (8 in extended)	8	8 to 32 (UTF-8: variable)
Characters	128 (256 extended)	256	Over 1 lakh (1,00,000+)
Languages	Mostly English	Mostly English	All world languages
Used in	PCs, internet text	IBM mainframes	Web, mobiles, all modern software

QUICK REVISION

✓ Character representation = converting characters to binary codes.

✓ n bits → 2^n characters.

✓ BCD: 4 bits per decimal digit; ASCII: 7 bits; EBCDIC: 8 bits; Unicode: all languages.

CHAPTER 10

1's Complement, 2's Complement and N's Complement

Negative numbers and easy subtraction

DEFINITION

A **complement** of a number is another number that, when **combined with the original**, gives a fixed result (like the largest digit-number or the next power of the base). Complements are used to **represent negative numbers** and to **perform subtraction by addition**.

The ALU of a computer needs only an **adder** circuit if subtraction is done using complements. This makes computer hardware simpler and cheaper.

10.1 Two Types of Complement in Any Base

Let the base (radix) of a number system be **N** (also written r) and the number have **n** digits.

Complement	Formula	Decimal (N=10)	Binary (N=2)	Octal (N=8)	Hex (N=16)
(N−1)'s complement	$(N^n - 1) - \text{number}$	9's complement	1's complement	7's complement	15's (F's) complement
N's complement	$N^n - \text{number}$	10's complement	2's complement	8's complement	16's complement

DEFINITION

(N−1)'s complement of a number is found by subtracting **each digit** from the **largest digit of the base (N−1)**.
N's complement = (N−1)'s complement + **1**.

10.2 Decimal Complements (9's and 10's)

EXAMPLES

Find the 9's and 10's complement of 546700.
9's complement: subtract every digit from 9 → 9−5, 9−4, 9−6, 9−7, 9−0, 9−0 = **453299**.
10's complement = 9's complement + 1 = **453300**.

Find the 10's complement of 3456.
9's complement = 6543; add 1 → **6544**. (Check: 10000 − 3456 = 6544 ✓)

10.3 1's Complement (Binary)

DEFINITION

The **1's complement** of a binary number is obtained by **changing every 0 to 1 and every 1 to 0** (inverting all bits).

EXAMPLE
1's complement of **10110100** = **01001011**
1's complement of **01101** = **10010**

In a computer this is simply the **NOT** operation on every bit.

10.4 2's Complement (Binary)

DEFINITION
The **2's complement** of a binary number is obtained by finding its **1's complement and adding 1** to the result.

Method 1 (Invert and add 1):

EXAMPLE
Find the 2's complement of **0101**:
Step 1: 1's complement of 0101 = **1010**
Step 2: Add 1 → 1010 + 1 = **1011**
So 2's complement of 0101 is **1011**.

Method 2 (Shortcut): Start from the **right**. Copy all bits **up to and including the first 1**. Then **invert all remaining bits** on the left.

EXAMPLE
2's complement of **01101100**:
From the right: 0, 0, then first 1 → copy **100**
Invert the rest **01101** → **10010**
Answer: **10010100**
(Check by method 1: 1's = 10010011, add 1 → 10010100 ✓)

10.5 N's Complement in Other Bases (Octal and Hexadecimal)

EXAMPLES
Octal: number 25_8.
7's complement = (7−2)(7−5) = **52**.
8's complement = 52 + 1 = **53**. (Check: $8^2 - 25_8 = 64 - 21 = 43 = 53_8$ ✓)

Hexadecimal: number $2A_{16}$.
15's (F's) complement = (F−2)(F−A) = **D5**.
16's complement = D5 + 1 = **D6**. (Check: 256 − 42 = 214 = D6 ✓)

10.6 Representing Negative Numbers (Signed Numbers)

To store negative numbers, the **leftmost bit** is used as a **sign bit** (MSB): **0 = positive, 1 = negative**. Three methods exist:

Method	How −5 is formed (4 bits)	−5	Range (n bits)
Sign-magnitude	Sign bit 1 + magnitude 101	1101	$-(2^{n-1}-1)$ to $+(2^{n-1}-1)$
1's complement	Invert all bits of +5 (0101)	1010	$-(2^{n-1}-1)$ to $+(2^{n-1}-1)$
2's complement	Invert bits of +5 and add 1	1011	-2^{n-1} to $+(2^{n-1}-1)$

REMEMBER

2's complement is used in almost all modern computers because it has only **one zero** (sign-magnitude and 1's complement have both +0 and −0) and the addition circuit works the same for positive and negative numbers.

WORKED EXAMPLES

8-bit range in 2's complement: −128 to +127 (that is -2^7 to 2^7-1).

Store −5 in 8 bits: +5 = 00000101 → invert = 11111010 → add 1 = **11111011**.

Read 11110110: MSB = 1, so it is negative. Take its 2's complement: 00001001 + 1 = 00001010 = 10. So the value is **−10**.

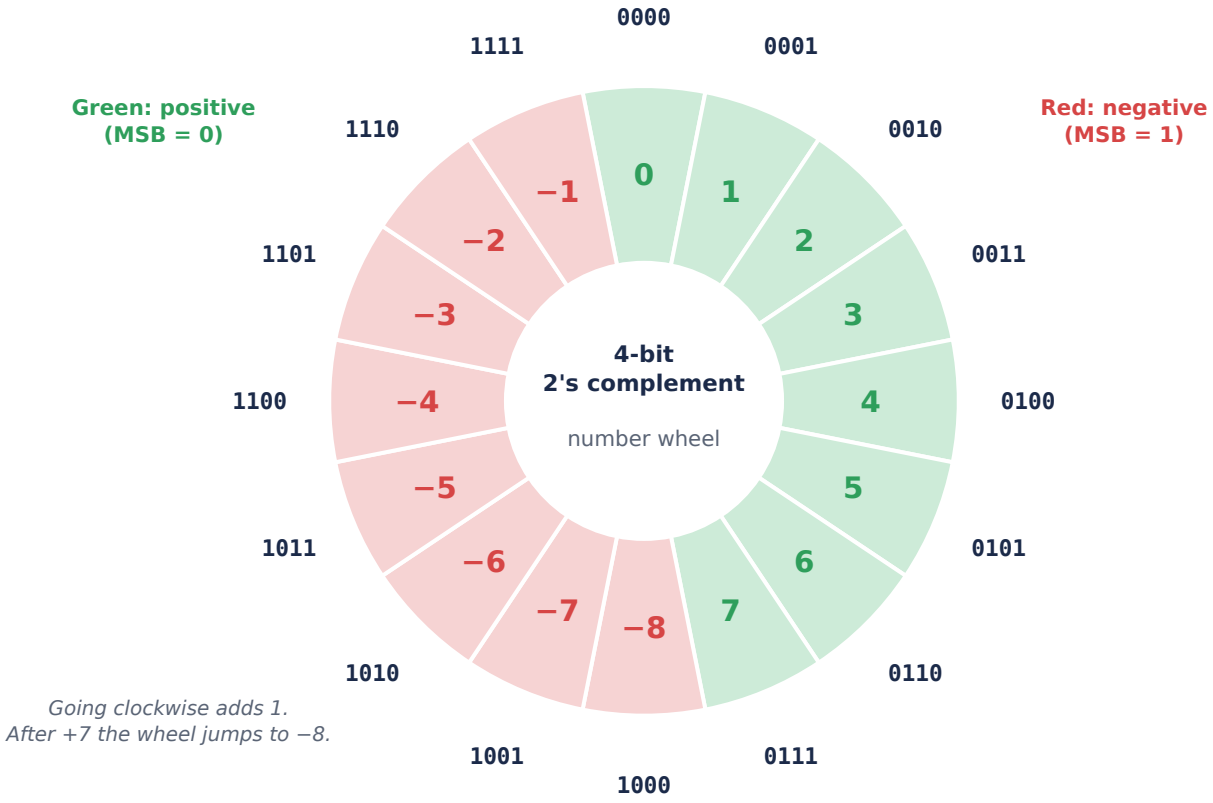


Figure 10.1 – 4-bit 2's complement values arranged on a wheel

All 4-bit values in 2's complement

Binary	Value	Binary	Value
0000	0	1000	-8
0001	1	1001	-7
0010	2	1010	-6
0011	3	1011	-5
0100	4	1100	-4
0101	5	1101	-3
0110	6	1110	-2
0111	7	1111	-1

10.7 Subtraction Using Complements (Decimal Idea)

We can replace **A − B** by **A + (complement of B)**. Look at the decimal idea first:

EXAMPLES

72 − 35 using 10's complement

10's complement of 35 = 99 − 35 + 1 = 65.

72 + 65 = 137. Ignore (discard) the carry 1 → **37** ✓

72 − 35 using 9's complement

9's complement of 35 = 64.

72 + 64 = 136. Add the end-around carry: 36 + 1 = **37** ✓

35 − 72 (answer is negative)

10's complement of 72 = 28.

35 + 28 = 63 (no carry). No carry means the answer is negative: take the 10's complement of 63 = 37 → **−37** ✓

The same rules are applied to binary numbers in the next chapter.

QUICK REVISION

- ✓ (N−1)'s complement: subtract each digit from N−1. N's complement = (N−1)'s complement + 1.
- ✓ 1's complement: flip all bits. 2's complement: flip all bits + 1.
- ✓ Negative numbers are stored in 2's complement; MSB 1 means negative.
- ✓ n-bit 2's complement range: -2^{n-1} to $+2^{n-1}-1$.

CHAPTER 11

Binary Arithmetic

Addition, Subtraction and Division of binary numbers

DEFINITION

Binary arithmetic means performing arithmetic operations (addition, subtraction, multiplication and division) on numbers written in the binary system, using only the digits 0 and 1.

The rules are just like decimal arithmetic – but because there are only two digits, they are very small and simple. The ALU of a computer performs these operations using electronic circuits.

11.1 Binary Addition

DEFINITION

Binary addition is the process of adding two or more binary numbers according to four basic rules. When the sum is 2 (binary 10), we write **0 and carry 1** to the next column.

Rule	Result	Remark
0 + 0	0	carry 0
0 + 1	1	carry 0
1 + 0	1	carry 0
1 + 1	10	write 0, carry 1
1 + 1 + 1	11	write 1, carry 1 (when a carry comes in)

EXAMPLE 1

Add 1011 + 0110 (that is 11 + 6):

```
  Carry    1 1 1 . .
             1 0 1 1
  +          0 1 1 0
           ---------
  Sum      1 0 0 0 1
```

Column by column (from right): 1+0 = 1; 1+1 = 0 carry 1; 0+1+1 = 0 carry 1; 1+0+1 = 0 carry 1; last carry 1 comes down. Answer $\mathbf{10001_2 = 17}$ ✓ (11 + 6 = 17).

EXAMPLE 2

Add 10110 + 1101 (that is 22 + 13):

```
  Carry    1 1 1 . . .
             1 0 1 1 0
  +          0 1 1 0 1
           -----------
  Sum      1 0 0 0 1 1
```

Answer $\mathbf{100011_2 = 35}$ ✓ (22 + 13 = 35).

11.2 Binary Subtraction

DEFINITION

Binary subtraction is the process of subtracting one binary number from another using four rules. When 1 is subtracted from 0, we **borrow 1** from the next left column (the borrowed 1 is worth 2 in this column).

Rule	Result	Remark
0 − 0	0	
1 − 0	1	
1 − 1	0	
0 − 1	1	borrow 1 from the left column (10 − 1 = 1)

(a) Direct method (with borrow)

Example: 10110 − 01011 (that is 22 − 11).

Bit position	Minuend bit	Subtrahend bit	Borrow in	Difference	Borrow out
0	0	1	0	1	1
1	1	1	1	1	1
2	1	0	1	0	0
3	0	1	0	1	1
4	1	0	1	0	0

Reading the difference column from the top row (right-most bit) upward gives the answer **01011_2 = 11** ✓.

(b) Subtraction using 1's complement

Steps: (1) Find the 1's complement of the number to be subtracted. (2) Add it to the first number. (3) If there is a **carry** out, add it back to the result (**end-around carry**) – the answer is positive. (4) If there is **no carry**, the answer is negative – take the 1's complement of the result.

EXAMPLES

1101 − 0101 (13 − 5): 1's complement of 0101 = 1010.
1101 + 1010 = **1**0111 (carry 1).
End-around carry: 0111 + 1 = **1000** = **8** ✓ (positive).

0101 − 1101 (5 − 13): 1's complement of 1101 = 0010.
0101 + 0010 = 0111 (no carry).
No carry → negative. 1's complement of 0111 = 1000 → **−8** ✓.

(c) Subtraction using 2's complement

Steps: (1) Find the 2's complement of the number to be subtracted. (2) Add it to the first number. (3) If there is a **carry**, **discard** it – the answer is positive. (4) If there is **no carry**, the answer is negative – take the 2's complement of the result.

EXAMPLES

1101 − 0101 (13 − 5): 2's complement of 0101 = 1011.
1101 + 1011 = **1**1000 (carry 1).
Discard the carry → **1000** = **8** ✓.

0101 − 1101 (5 − 13): 2's complement of 1101 = 0011.
0101 + 0011 = 1000 (no carry).
No carry → negative. 2's complement of 1000 = 1000 (= 8) → **−8** ✓.

REMEMBER

Because **2's complement** needs no end-around carry, it is the method used inside real computers.

11.3 Binary Multiplication (Short Note)

DEFINITION

Binary multiplication follows the rules 0×0 = 0, 0×1 = 0, 1×0 = 0, 1×1 = 1. It is done by shifting and adding, like long multiplication in decimal.

EXAMPLE: 101 × 11

```
      1 0 1      (5)
x     1 1        (3)
---------
      1 0 1      (101 x 1)
1 0 1 0          (101 x 1, shifted left by 1)
---------
1 1 1 1        = 15  ✓
```

11.4 Binary Division

DEFINITION

Binary division is the process of finding how many times one binary number (divisor) is contained in another (dividend). It uses **long division** with repeated **comparison and subtraction**, as in decimal.

Method: Take bits of the dividend from the left, one by one. If the current value is **greater than or equal to the divisor**, write **1** in the quotient and subtract the divisor. Otherwise write **0** and bring down the next bit. The number left at the end is the **remainder**.

REMEMBER

Division by zero (0) is **not defined**. Also, dividend = divisor × quotient + remainder.

Example 1: 11110 ÷ 101 (30 ÷ 5)

Step	Bit brought down	Current value	Compare with 101	Quotient bit	Remainder
1	1	1	1 < 101 → cannot subtract	0	1
2	1	11	11 < 101 → cannot subtract	0	11
3	1	111	111 ≥ 101 → subtract	1	10
4	1	101	101 ≥ 101 → subtract	1	0
5	0	0	0 < 101 → cannot subtract	0	0

Quotient = **110_2 = 6**, Remainder = **0 = 0**. Check: 101 (5) × 110 (6) = 11110 (30) ✓.

Example 2: 1101 ÷ 11 (13 ÷ 3)

Step	Bit brought down	Current value	Compare with 11	Quotient bit	Remainder
1	1	1	1 < 11 → cannot subtract	0	1
2	1	11	11 ≥ 11 → subtract	1	0
3	0	0	0 < 11 → cannot subtract	0	0
4	1	1	1 < 11 → cannot subtract	0	1

Quotient = **100_2 = 4** and Remainder = **1_2 = 1**. Check: 3 × 4 + 1 = 13 ✓.

QUICK REVISION

- ✓ Addition: 1+1 = 10 (write 0, carry 1); 1+1+1 = 11.
- ✓ Subtraction: 0−1 needs a borrow.
- ✓ Subtraction by complement: A − B = A + complement of B. With a carry → positive; no carry → negative (complement the result).
- ✓ Division: compare, write 1 or 0, subtract, bring down the next bit.

CHAPTER 12

ASCII Code

American Standard Code for Information Interchange

DEFINITION

ASCII (American Standard Code for Information Interchange) is a **character encoding standard** that gives a unique **7-bit** number (0 to 127) to every English letter, digit, punctuation mark and control command, so that different computers can exchange text correctly.

ASCII was developed in the early 1960s. As it uses 7 bits, it can represent 2^7 = **128 characters**. In a computer, each ASCII character is stored in one **byte** (the 8th bit is 0, or was used earlier as a parity bit).



How a computer handles one character

Figure 12.1 – How a character travels inside the computer

12.1 Structure of ASCII

Decimal range	Type	Examples
0 – 31 and 127	Control characters (not printable)	NUL (0), BEL (7), Backspace (8), TAB (9), Line Feed (10), Enter / CR (13), ESC (27), DEL (127)
32	Space	Blank space
33 – 47, 58 – 64, 91 – 96, 123 – 126	Punctuation & symbols	! " # $ % & ( ) * + , - . / : ; < = > ? @ [ ] { } ~
48 – 57	Digits 0 – 9	'0' = 48, '9' = 57
65 – 90	Capital letters A – Z	'A' = 65, 'Z' = 90
97 – 122	Small letters a – z	'a' = 97, 'z' = 122

REMEMBER

Easy to remember: A = 65, a = 97, 0 = 48. Small letter = Capital letter + **32** (for example a = 65 + 32 = 97). The digit character '7' has code 48 + 7 = 55; it is not the number 7.

12.2 ASCII Table of Letters and Digits

Capital letters

Char	Dec	Binary (7-bit)
A	65	1000001
B	66	1000010
C	67	1000011
D	68	1000100
E	69	1000101
F	70	1000110
G	71	1000111
H	72	1001000
I	73	1001001
J	74	1001010
K	75	1001011
L	76	1001100
M	77	1001101

Char	Dec	Binary (7-bit)
N	78	1001110
O	79	1001111
P	80	1010000
Q	81	1010001
R	82	1010010
S	83	1010011
T	84	1010100
U	85	1010101
V	86	1010110
W	87	1010111
X	88	1011000
Y	89	1011001
Z	90	1011010

Small letters

Char	Dec	Binary (7-bit)
a	97	1100001
b	98	1100010
c	99	1100011
d	100	1100100
e	101	1100101
f	102	1100110
g	103	1100111
h	104	1101000
i	105	1101001
j	106	1101010
k	107	1101011
l	108	1101100
m	109	1101101

Char	Dec	Binary (7-bit)
n	110	1101110
o	111	1101111
p	112	1110000
q	113	1110001
r	114	1110010
s	115	1110011
t	116	1110100
u	117	1110101
v	118	1110110
w	119	1110111
x	120	1111000
y	121	1111001
z	122	1111010

Digits

Char	Dec	Binary (7-bit)
0	48	0110000
1	49	0110001
2	50	0110010
3	51	0110011
4	52	0110100

Char	Dec	Binary (7-bit)
5	53	0110101
6	54	0110110
7	55	0110111
8	56	0111000
9	57	0111001

12.3 Worked Examples

EXAMPLE 1

Write “Hi!” in ASCII.

Character	ASCII (decimal)	Binary (8-bit)
H	72	01001000
i	105	01101001
!	33	00100001

So “Hi!” is stored as **01001000 01101001 00100001** – 3 bytes.

EXAMPLE 2

Find the ASCII code of “CAT”.

C = 67, A = 65, T = 84 → in binary: **01000011 01000001 01010100**.

EXAMPLE 3

Decode 01001111 01001011.

01001111 = 79 = 'O'; 01001011 = 75 = 'K' → the message is **OK**.

EXAMPLE 4

Convert capital to small letter. 'G' = 71. Small 'g' = 71 + 32 = **103**. In binary, G = 01000111 and g = 01100111 – only **one bit** (the 6th bit) is different.

12.4 Extended ASCII and Limitations

- **Extended ASCII** uses all **8 bits** (codes 128 – 255) giving 256 characters. It adds accented letters, line-drawing symbols and others.
- ASCII cannot show characters of Hindi, Chinese or Arabic. For this, **Unicode** was developed. The first 128 Unicode characters are the same as ASCII.

QUICK REVISION

✓ ASCII = 7-bit code, 128 characters (0–127).

✓ A = 65, a = 97, 0 = 48, space = 32.

✓ Small letter = Capital letter + 32.

✓ Extended ASCII = 8 bits, 256 characters.

CHAPTER 13

Hardware and Software

The body and the mind of a computer

DEFINITION

Hardware means all the **physical parts** of a computer that we can see and touch.

DEFINITION

Software is a set of **programs and instructions** that tells the hardware what to do. We cannot touch software.

EXAMPLE

Think of a human body and the mind. The **body is hardware** and the **mind (thoughts, knowledge) is software**. A body without a mind cannot work, and a mind without a body cannot act. In the same way, a computer needs **both**.

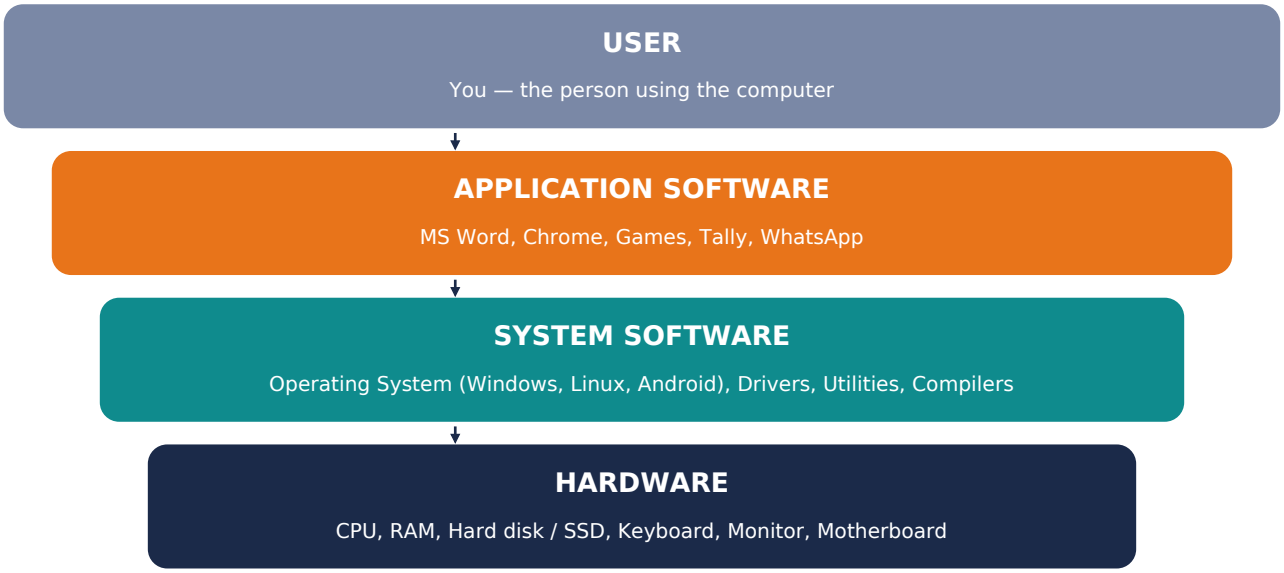


Figure 13.1 – Layers of a computer system

13.1 Hardware

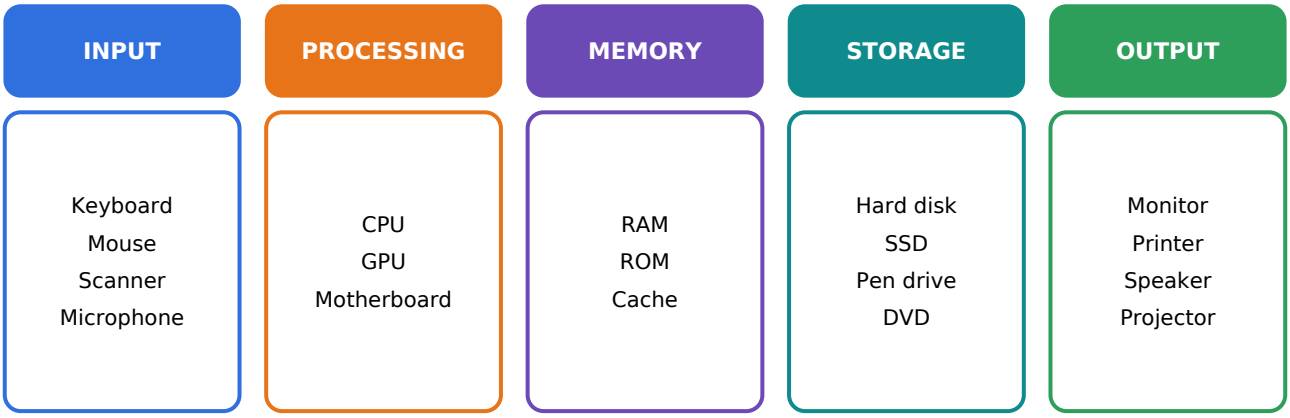


Figure 13.2 – Main categories of hardware

(a) Input devices

Definition: Devices used to give data and commands to the computer. *Examples:* keyboard, mouse, scanner, microphone, webcam, touch screen, barcode reader, joystick.

(b) Output devices

Definition: Devices that show the results of processing. *Examples:* monitor, printer, speaker, projector, plotter.

(c) Processing devices

Definition: The parts that perform calculations and control. The **CPU** (processor) does the main work; the **GPU** handles graphics; the **motherboard** is the main circuit board that connects everything.

(d) Memory

- **RAM** (Random Access Memory): temporary, fast working memory; **volatile** – data is lost when power is off.
- **ROM** (Read Only Memory): permanent memory that stores start-up instructions (BIOS); data stays when power is off.
- **Cache**: very fast small memory close to the CPU that keeps frequently used data.

(e) Storage devices

Definition: Devices that store data permanently. *Examples:* hard disk (HDD), solid state drive (SSD), pen drive, memory card, CD/DVD.

13.2 Software

Type	Meaning	Examples
System software	Controls the computer hardware and provides a platform for other software	Operating system (Windows, Linux, macOS, Android), device drivers, utility programs, language processors, firmware
Application software	Helps the user to do a specific task	MS Word, Excel, Chrome, VLC, Tally, games, WhatsApp
Utility software	Maintains and protects the computer	Antivirus, disk cleanup, backup, file compression (WinZip)

System software

DEFINITION
System software is software that manages and controls the computer hardware and provides a base on which application software can run.

- **Operating System (OS):** manages memory, files, processes and devices (e.g. Windows, Linux, Android).
- **Device drivers:** small programs that help the OS talk to devices such as printers.
- **Language processors:** assembler, compiler and interpreter (Chapter 15).
- **Firmware:** permanent software stored in a chip, such as the BIOS.

Application software

DEFINITION
Application software is software designed to perform specific tasks for the user.

General-purpose application: can be used for many jobs (word processor, spreadsheet, browser). **Specific-purpose** application: made for one job (railway reservation, payroll, hospital management). **Custom (bespoke)** software is made specially for one customer.

13.3 Hardware vs Software

Hardware	Software
Physical parts that can be touched	Programs; cannot be touched
Made of electronic and mechanical parts	Written by programmers in a language
Can be damaged by dust, heat, water	Can be corrupted by viruses and bugs
Examples: CPU, RAM, keyboard, monitor	Examples: Windows, MS Word, Chrome
Needs software to work	Needs hardware to run on

QUICK REVISION
✓ Hardware = physical parts; software = programs.
✓ Hardware types: input, output, processing, memory, storage.
✓ Software types: system software (OS, drivers, utilities, language processors) and application software.
✓ RAM is volatile; ROM and storage devices are non-volatile.

CHAPTER 14

Concept of Booting

POST and loading of the Operating System

DEFINITION

Booting is the process of **starting a computer**, in which the hardware is checked and the **operating system is loaded** from the disk into main memory (RAM) so that the computer becomes ready to use.

The word comes from **“bootstrapping”** – the saying “to pull yourself up by your own bootstraps”. A computer begins with an empty RAM, so it uses a tiny program stored in ROM to **load a bigger program (the OS)**, which then loads everything else.

14.1 Why is Booting Needed?

The CPU can execute only programs that are present in **RAM**. When the power is OFF, RAM is empty and the OS lies on the hard disk or SSD. Booting copies the OS from the disk into RAM.

14.2 Types of Booting

Type	Meaning	Example
Cold boot (hard boot)	Starting the computer when it is completely OFF by pressing the power button	Turning on a computer in the morning
Warm boot (soft boot / reboot)	Restarting a computer that is already ON, without switching power off	Clicking “Restart” or pressing Ctrl+Alt+Del

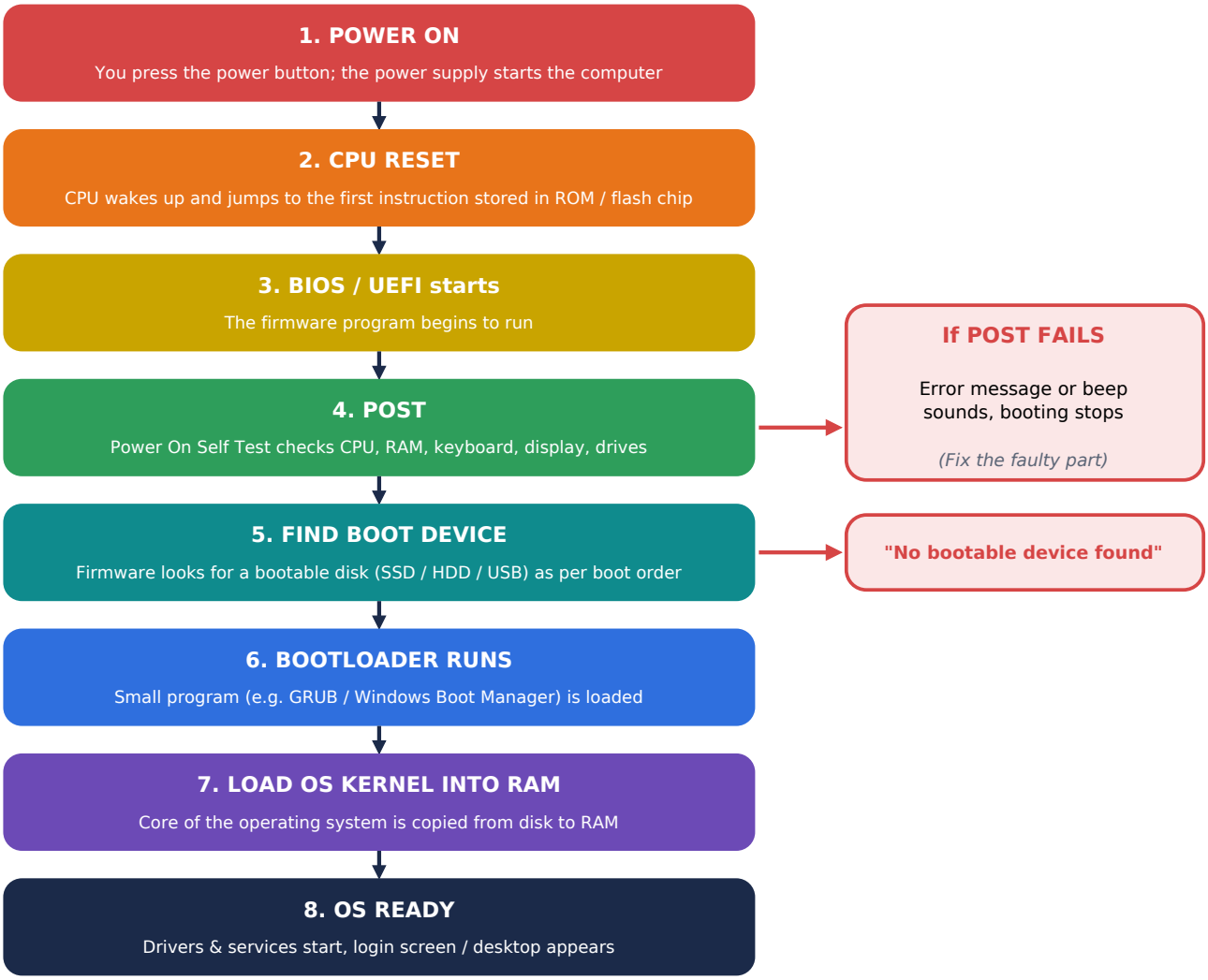


Figure 14.1 – Steps of the booting process

14.3 Steps of Booting

Step 1: Power on and CPU reset

When you press the power button, the **power supply (SMPS)** supplies electricity. The CPU resets and automatically starts executing the first instruction stored in the **ROM / flash memory** chip on the motherboard.

Step 2: BIOS / UEFI starts

DEFINITION

BIOS (Basic Input Output System) is **firmware** stored in a ROM/flash chip on the motherboard. It tests hardware and starts the boot process. Modern computers use its newer version, **UEFI** (Unified Extensible Firmware Interface), which is faster and more secure.

Step 3: POST (Power On Self Test)

DEFINITION
POST (Power On Self Test) is a **diagnostic program** in the BIOS/UEFI that automatically **checks the main hardware parts** of the computer (CPU, RAM, keyboard, display adapter, storage drives) as soon as the computer is switched on, **before** the operating system starts.

- It checks that the **CPU** is working.
- It tests the **RAM** by reading and writing to it.
- It detects the **keyboard, mouse, video card** and **disk drives**.
- If everything is fine, many computers give a **single short beep** and the boot process goes on.
- If a part is faulty, POST shows an **error message** or gives a **series of beeps** (the beep pattern varies by manufacturer) and booting stops.

EXAMPLE
A computer with a loose RAM stick may give repeated beeps and show a blank screen. After the RAM is fixed properly, POST passes and the computer starts normally.

Step 4: Finding the boot device

The BIOS/UEFI settings (stored in a small battery-powered **CMOS** memory) contain the **boot order**, for example: 1) SSD, 2) USB drive, 3) DVD. The firmware looks for a bootable device in this order. If none is found you see a message like **“No bootable device found”**.

Step 5: Bootloader

DEFINITION
A **bootloader** is a small program stored at the start of the boot disk (in the boot sector/EFI partition). Its job is to **find and load the OS kernel into RAM**.

EXAMPLE
GRUB (Linux), Windows Boot Manager.

Step 6: Loading of the Operating System

DEFINITION
Loading the OS means copying the **kernel** (the core part of the operating system) from the disk into RAM and starting it. The kernel then initialises **device drivers**, starts **system services** and finally shows the **login screen or desktop**.

REMEMBER
Order to remember: **Power ON → BIOS/UEFI → POST → Bootloader → OS kernel loaded into RAM → Desktop ready.**

14.4 BIOS vs UEFI

BIOS	UEFI
Older firmware	Newer, modern replacement of BIOS
Text-based setup screen, keyboard only	Graphical setup screen with mouse support
Slower start-up, works with smaller disks (MBR)	Faster start-up, supports very large disks (GPT) and Secure Boot

QUICK REVISION

✓ Booting = starting the computer and loading the OS into RAM.

✓ Cold boot = from power off; warm boot = restart.

✓ POST = Power On Self Test, checks hardware before OS loads.

✓ Sequence: Power on → BIOS/UEFI → POST → bootloader → kernel in RAM → desktop.

CHAPTER 15

Language Processors

Assembler, Compiler and Interpreter

15.1 Types of Programming Languages

Higher level = easier for humans, needs translation | Lower level = closer to the machine

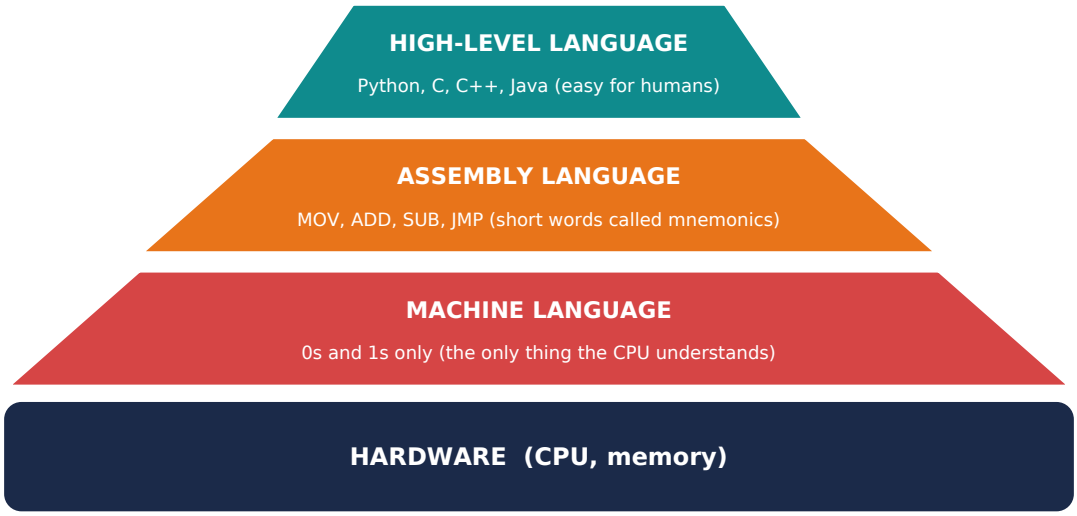


Figure 15.1 – Levels of programming languages

DEFINITION

Machine language is the **lowest-level language**, written only in **0s and 1s**. It is the only language the CPU understands directly. It is very fast, but very hard for humans to write and remember.

EXAMPLE

10110000 00000101 is a machine instruction for the x86 processor (it means: put the number 5 into register AL).

DEFINITION

Assembly language is a **low-level language** that uses short English-like words called **mnemonics** in place of binary codes.

EXAMPLE

MOV AL, 5
ADD AL, 3
(mnemonics: MOV = move, ADD = add)

DEFINITION
High-level language is a language that is **close to human language** (English and mathematics). It is easy to write, read and is **machine independent**.

EXAMPLE
Python: print("Hello"), C: printf("Hello"), Java, C++.

15.2 Language Processor (Translator)

DEFINITION
A **language processor** (translator) is a **system software** that **converts a program written in one language into another language** that the computer can execute. The program we write is called the **source code**; the translated program is the **object code** (machine code).

Three types of language processors are: **Assembler, Compiler** and **Interpreter**.

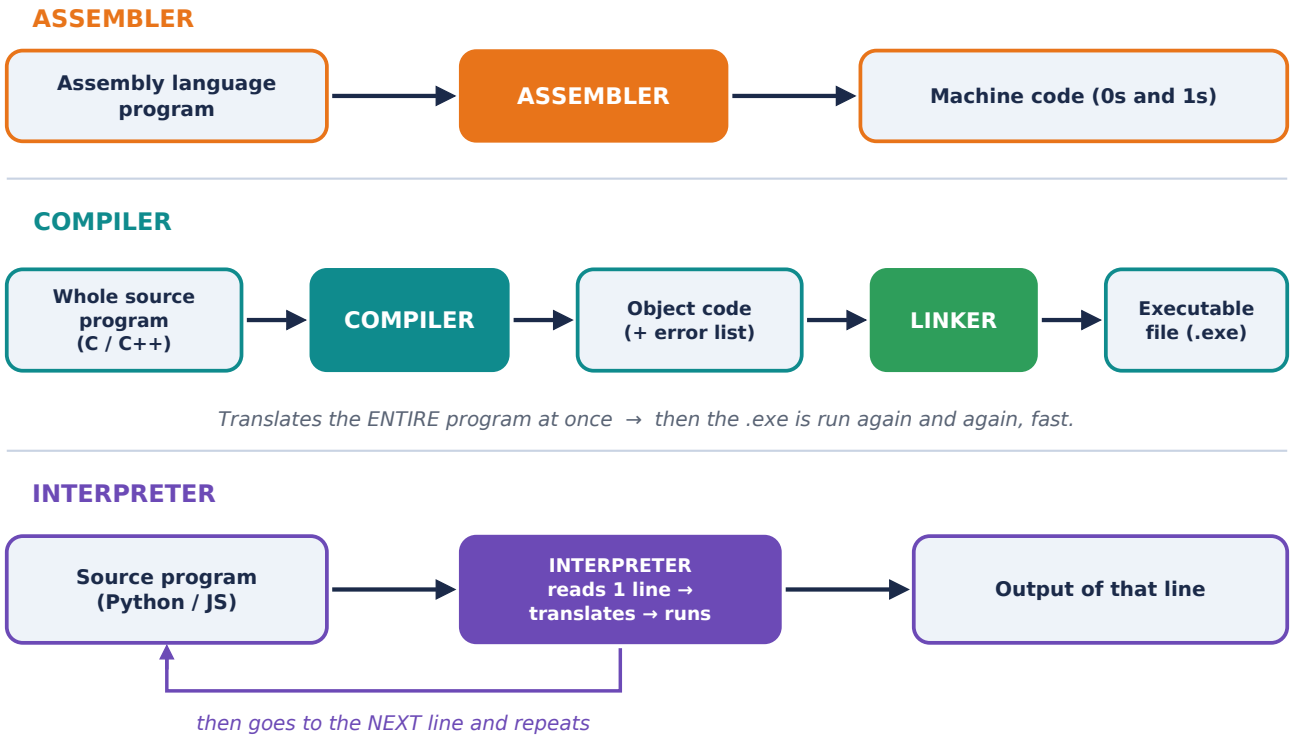


Figure 15.2 – How the assembler, compiler and interpreter work

15.3 Assembler

DEFINITION
An **assembler** is a language processor that **translates an assembly language program into machine language**.

- It replaces mnemonics (like MOV) by their binary **opcodes** and names (labels) by memory addresses.

- It usually reads the program twice (**two-pass assembler**): first to note down labels, then to produce the code.
- It reports errors such as a wrong mnemonic.

EXAMPLE
Assembly: **MOV AL, 5** → Assembler → Machine code (hex) **B0 05** (binary 10110000 00000101).

15.4 Compiler

DEFINITION
A **compiler** is a language processor that **translates the whole high-level program into machine code at one time**, before the program is run. It also gives a list of all errors found in the program.

- Output is a separate **executable file** (.exe) that can be run any number of times **without the compiler**.
- **Execution is fast** because translation is already done.
- Detects all syntax errors together and shows an **error list**.
- The program is not run until all errors are removed.

EXAMPLE
Languages normally using a compiler: **C, C++, Pascal, Go, Rust**. Compiler example: GCC. Steps: Source code → Compiler → Object code → Linker → Executable file → Run.

15.5 Interpreter

DEFINITION
An **interpreter** is a language processor that **translates and executes a program line by line**. It reads one statement, converts it, runs it, and then moves to the next.

- No separate machine-code file is created.
- The interpreter is needed **every time** the program runs.
- Easy to find and correct errors, because it **stops at the line where the error occurs**.
- Execution is **slower** than a compiled program.

EXAMPLE
Languages normally using an interpreter: **Python, JavaScript, Ruby, PHP, classic BASIC**. Java is a mix: the compiler first makes **bytecode**, and the Java Virtual Machine then interprets/executes it. Python also first makes bytecode internally.

EXAMPLE

Suppose a program has:
Line 1: PRINT “Hello”
Line 2: PRINT “World (missing closing quote)
A **line-by-line interpreter** (like classic BASIC) prints **Hello** and then shows an error at line 2. A **compiler** finds the error before running anything and produces no executable until it is corrected.

15.6 Compiler vs Interpreter vs Assembler

Point	Assembler	Compiler	Interpreter
Input	Assembly language	High-level language	High-level language
Output	Machine code	Machine code (executable file)	No separate file – result directly
Translation	Whole program (2 passes)	Whole program at once	Line by line
Speed of running	Fast	Fast	Slower
Error reporting	Reports errors of assembly code	Shows all errors after scanning the program	Stops at the first error line
Memory needed	Less	More (object code stored)	Less
Examples	NASM, MASM	GCC (C/C++), javac	Python, JavaScript, Ruby

QUICK REVISION

✓ Machine language = 0s and 1s; assembly = mnemonics; high-level = human-like.
✓ Language processor converts source code into object/machine code.
✓ Assembler: assembly → machine code.
✓ Compiler: whole program at once, fast to run, error list.
✓ Interpreter: line by line, slower, stops at first error.

CHAPTER 16

Quick Reference Sheet

Handy tables for last-minute revision

16.1 Powers of 2

n	0	1	2	3	4	5	6	7	8	9	10	11	12	16
2^n	1	2	4	8	16	32	64	128	256	512	1024	2048	4096	65536

16.2 Number Systems at a Glance

System	Base	Weights	Bits per digit
Binary	2	... 16, 8, 4, 2, 1	1
Octal	8	... 512, 64, 8, 1	3
Decimal	10	... 1000, 100, 10, 1	–
Hexadecimal	16	... 4096, 256, 16, 1	4

16.3 Complement Rules

Number system	(N−1)'s complement	N's complement
Binary	1's: flip bits	2's: flip bits + 1
Octal	7's: 7 − each digit	8's: 7's + 1
Decimal	9's: 9 − each digit	10's: 9's + 1
Hexadecimal	F's: 15 − each digit	16's: F's + 1

16.3 Important ASCII Values

Character	Space	0	9	A	Z	a	z	Enter	Esc
Decimal	32	48	57	65	90	97	122	13	27

16.4 Boot Sequence in One Line

REMEMBER

Power ON → BIOS/UEFI → **POST** → Boot device → Bootloader → OS kernel loaded into RAM → Desktop.

Thank you for reading!

Keep practising the conversions and complements – they are the most scoring topics.

Made by Subhadeep Paul